CallMiner Bulk Export User Guide

VERSION 2025.11

"CallMiner®", "Eureka®", "TopicMiner®", "myEureka", "EurekaLive", "Eureka Coach", "Eureka Redact", "Eureka API", "Eureka RealTime", the "CallMiner Eureka" logo, the "CallMiner myEureka" logo, the "CallMiner EurekaLive" logo, "Listen to your customers. Improve your business.", "Illuminate", "Feedback Is A Gift", "Listen", "Engagement Optimization", the "Engagement Optimization" logo, "EO", "ORECX", "OREKA", and "OrecX OPEN SOURCE RECORDING" are trademarks or registered trademarks of CallMiner, Inc. in the United States and foreign jurisdictions. Other product names mentioned herein may be the trademarks of their respective owners.

© Copyright, 2008 - 2025, CallMiner, Inc.

All rights reserved. No part of this publication can be reproduced or used in any form or by means - graphic, electronic, or mechanical, including photocopying, recording, taping, or information storage and retrieval systems - without permission of CallMiner, Inc.

CONTENTS

1 Introduction **7**
1. 1 | Intended Audience 7
1. 2 | Requirements 8
1. 3 | Feedback 8
1. 4 | What's New in This Version 9
2025.11 9
2 Managing Exports Through the API **10**
2. 1 | Log In and Authenticate with the API 11
Log In to the API Interactive Documentation 11
Authenticate when Using a Third-Party Application 12
2. 2 | Create a Job to Export Contact Data 15
Using the Interactive Documentation 15
Using a Third-Party Application 15
The Response 30
2. 3 | Retrieve a List of All of Your Export Jobs 32
Using the Interactive Documentation 32
Using a Third-Party Application 32
The Response 32
2. 4 | Retrieve a Specific Export Job 34
Using the Interactive Documentation 34
Using a Third-Party Application 34
The Response 34
2. 5 | Update a Recurring Export Job 36
Using the Interactive Documentation 36
Using a Third-Party Application 36

The Response 43
2. 6 | Delete One of Your Export Jobs 44
Using the Interactive Documentation 44
Using a Third-Party Application 44
The Response 45
2. 7 | Retrieve the History of Data Exports for All Export Jobs 46
Using the Interactive Documentation 46
Using a Third-Party Application 46
The Response 47
2. 8 | Download the Files from a Specific Export 49
Using the Interactive Documentation 50
Using a Third-Party Application 50
The Response 51
2. 9 | Delete an Export History Record 53
Using the Interactive Documentation 53
Using a Third-Party Application 53
The Response 54
Appendix A CallMiner Bulk Export API URLs 55
Interactive Documentation URL 55
URL for Third-Party Application Use 55
Appendix B CallMiner's Identity Provider Service URLs 56
Appendix C Managing Third-Party Storage Targets 57
Create or Update a Third-Party Storage Target 58
Using the Interactive Documentation 59
Using a Third-Party Application 59
Amazon AWS S3 60
The Response 61
Retrieve Existing Storage Target Information 62

Using the Interactive Documentation 63
Using a Third-Party Application 63
The Response 64
Amazon AWS S3 64
Delete a Third-Party Storage Target from Bulk Export 66
Using the Interactive Documentation 67
Using a Third-Party Application 67
The Response 68
Appendix D Export Data Types 69
Appendix E JSON-Formatted Schema File Contents 87
Appendix F Retrieve Metadata Information 89
Using the Interactive Documentation 90
Using a Third-Party Application 90
The Response 90
Appendix G SearchFilters Basics 92
Filter Options 92
Operator Options 100
Appendix H Retrieve Content SearchFilters Values 109
Using the Interactive Documentation 110
Using a Third-Party Application 110
The Response 111
Categories 111
Tags 113
Sections 113
Scores 114
Alerts 117
Appendix I Quartz Cron Basics 118
Special Character Meanings 120

More Examples 122
Appendix J Best Practices and FAQs 124
My export is too large and it's affecting system performance... 124
Handling updated versions of contacts... 125
Combining all data into one table... 125
Bringing data into Snowflake or other data storage systems... 126
Scaling with large volumes of data... 126
Handling duplicate contact exports caused by retroprocessing... 128
Appendix K HTTP Status Codes 130
Appendix L Troubleshooting 131
Received Error When Creating a New Export Job 132
Received Error on an Existing Recurring Export Job 132
Appendix M Migration Guide: Data API to Bulk Export API 136
Use Cases for Getting Information Out of Your Eureka System 136
Use Cases for Exporting Data Out of Your Eureka System 139

1 Introduction

The CallMiner Bulk Export API provides you with a quick, easy way to export all contact data out of your Eureka system. With it, you can:

- Export contact data, including transcripts, categories, scores, and much more in bulk to CSV. (See Export Data Types for a full list of the types of data that can be exported.) Along with the CSV file(s), you will also receive a JSON-formatted schema file for each data type that contains further information about your export. (For more information, see JSON-Formatted Schema File Contents.)

  > **Note**: When you receive your exported contact data, you will receive a main zip file that contains the JSON-formatted schema file(s) for each requested data type, and within that main zip file, you will find a zipped CSV file for each data type (containing the actual contact data).

- Export your chosen data one time immediately, or you can create a recurring job that will export and deliver your data on a continuous schedule.

If you're already familiar with the CallMiner Data API or the *CallMiner Data API Cookbook*, you'll find that we've taken some of our most commonly used bulk export use cases and made it even easier to export the contact data you're most interested in from your Eureka system--all with one easy-to-use Bulk Export API route. For a quick overview of how you can migrate your exports from the Data API to the Bulk Export API, see Migration Guide: Data API to Bulk Export API.

> **Note**: The Bulk Export API allows you to export all contact data in your system by default, regardless of the hierarchy alignment of your export user. To filter the data export, take advantage of `MetadataFilters` on your export requests.

Keep reading to learn how to use the Bulk Export API to export all of the contact data you care about on whatever delivery schedule works best for you.

1.1 | Intended Audience

This document is intended for users who wish to export their contact data from CallMiner in bulk for data warehousing use cases.

1. 2 | Requirements

To access and use the Bulk Export API, you'll need the following:

- The URL for the Bulk Export API or the URL for the Bulk Export API interactive documentation. If you don't already have it, see CallMiner Bulk Export API URLs.
- (*If using the interactive documentation*) Valid login credentials, including your realm name, email address, and password.

  > **Important**: You will use the same email address to log in to the Bulk Export API that you do for Analyze or Coach, but your password may be different.

- Certain permissions. We recommend that your user be aligned to the **Administrator** role in Analyze to be sure that you have permission to export whichever data types you want. If you do not have the required role or permissions, contact your system administrator.
- (*If using a third-party application to access the API*) A Client ID and Secret from support@callminer.com. You'll use this Client ID and Secret in the Log In and Authenticate with the API section to request an access token from CallMiner's Identity Provider Service that allows you to authenticate against the API.
- (*If using a third-party application to access the API*) The URL for CallMiner's Identity Provider Service. If you don't already have it, see CallMiner's Identity Provider Service URLs.
- (*If using a third-party storage provider*) A fully configured storage location for your chosen provider (for example Amazon AWS S3) where you want your Bulk Export data delivered. This storage target must be fully configured using the storage targets routes in the Bulk Export API before creating an export job.

1. 3 | Feedback

Your feedback plays an important part in improving our technical documentation. Please send any questions, comments, or suggestions for improvements to docfeedback@callminer.com.

1.4 | What's New in This Version

2025.11

- You cannot delete a storage target if one or more export jobs are delivering data to that storage target location. See Delete a Third-Party Storage Target from Bulk Export for more information.

2 Managing Exports Through the API

With the CallMiner Bulk Export API, you can export your contact data using two different methods, depending on your level of API experience:

- **BULK EXPORT API INTERACTIVE DOCUMENTATION** A web-based interface that allows you to export your contact data directly through our API and provides informative, interactive documentation on how to use the API to export your data. We recommend this method for beginner API users.
- **BULK EXPORT API USING A THIRD-PARTY APPLICATION** A way for more advanced API users to access and export contact data. CallMiner gives you the option to use whatever third-party API application you'd like to use to access our API and export your contact data.

Each section of this document explains how to export your contact data using either the Bulk Export API interactive documentation or the third-party application of your choice, so whichever method you'd like to use, we've got you covered.

Choose what you'd like to do from the list below and let's get started:

2. 1 | Log In and Authenticate with the API 11

2. 2 | Create a Job to Export Contact Data 15

2. 3 | Retrieve a List of All of Your Export Jobs 32

2. 4 | Retrieve a Specific Export Job 34

2. 5 | Update a Recurring Export Job 36

2. 6 | Delete One of Your Export Jobs 44

2. 7 | Retrieve the History of Data Exports for All Export Jobs 46

2. 8 | Download the Files from a Specific Export 49

2. 9 | Delete an Export History Record 53

2. 1 | Log In and Authenticate with the API

You can either use the Bulk Export API interactive documentation to send requests and manage your exports, or, if you're a more experienced API user, you can use the third-party API application of your choice to make requests.

Depending on which option you choose, you'll need to follow different login and authentication steps:

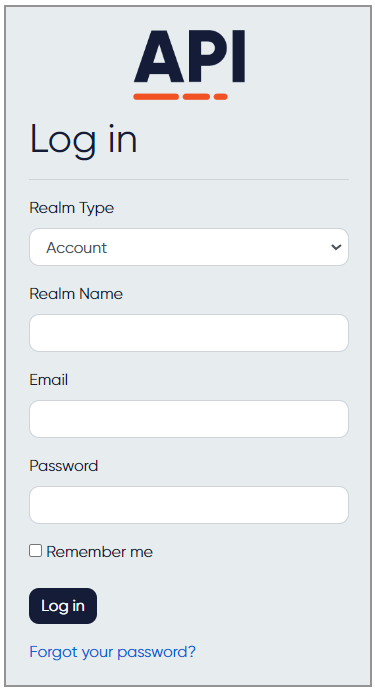
Log In to the API Interactive Documentation 11

Authenticate when Using a Third-Party Application 12

Log In to the API Interactive Documentation

To log in to the Bulk Export API interactive documentation:

1. In your browser, navigate to the applicable Bulk Export API interactive documentation URL. To find your URL, see CallMiner Bulk Export API URLs.
2. You'll be redirected to CallMiner's Identity Provider Service login page.



3. If it's not already selected for you, choose *Account* in the Realm Type drop-down and enter your Realm Name in the appropriate field. Your Realm Name is the company name that was provided to

CallMiner during the initial onboarding process.

> **Note**: If these fields are not visible in the Log in dialogue box, it means that they have already been set in the query parameters. Skip to the step below.

4. Enter your email address and password into the appropriate fields.

> **Important**: Your email address is the email address your organization provided to CallMiner when your Bulk Export account was created. It may be different than the one you use to log in to Analyze or Coach. You may request these credentials from support@callminer.com.

5. Click **Log in**.
6. You are now logged in to the Bulk Export API interactive documentation. You're ready to start exporting your data.

Authenticate when Using a Third-Party Application

This section describes how to use a CallMiner-provided Client ID and Secret to generate an access token from CallMiner's Identity Provider Service. Once you've created a token, you'll use this token in the `Authorization` header to authenticate each request you make against the CallMiner Bulk Export API.

> **Note**: Before you can request a token from CallMiner's Identity Provider Service, you'll need to request a Client ID and Secret from support@callminer.com.

To request an access token, complete the following steps:

1. Open the third-party application you'd like to use.
2. Enter the URL for CallMiner's Identity Provider Service. To find your URL, see CallMiner's Identity Provider Service URLs.
3. Create a `POST` request with the `connect/token` route.
4. Set the `Content-Type` request header to *application/x-www-form-urlencoded* as the value.
5. Enter your Client ID, Client Secret, grant type, and scope into the body of the request:
   - **client_id**
     Enter the Client ID you received from CallMiner.
   - **client_secret**
     Enter the Client Secret you received from CallMiner.

- **grant_type**
  Enter the text `client_credentials` as the value.
- **scope**
  Enter the text `https://callminer.net/auth/platform-bulkexport` as the value.

Below is an example of what the body of the request might look like:

```
POST connect/token
Content-Type: application/x-www-form-urlencoded

{
   "client_id": "abcd-1234",
   "client_secret": "abcd-1234",
   "grant_type": "client_credentials",
   "scope": "https://callminer.net/auth/platform-bulkexport"
}
```

6. Send the request. One of the following happens:

   - If an error occurs, a status code in the **400** or **500** range is returned. Refer to HTTP Status Codes for more information.
   - If the request is successful, the API server responds with a **200** HTTP status code and the access token you requested.

     Below is an example of what the body of the response might look like:

     ```
     {
        "access_token": "abcd123",
        "expires_in": 3600,
        "token_type": "Bearer",
        "scope": "https://callminer.net/auth/platform-bulkexport"
     }
     ```

     The response includes:

     - **access_token**
       The access token string as issued by the authorization server.
     - **expires_in**
       The number of seconds that the token is granted for.

       > **Important**: This authentication token has a lifespan of 1 hour.

- **token_type**
  The type of access token.
- **scope**
  The application scope that the token will allow you to access.

7. For the lifetime of this token, you will set the `Authorization` header of each request you make against the Bulk Export API to the word *Bearer* followed by a space and then the `access_token` value. For example:

```
Authorization: Bearer abcd123
```

> **Note**: To avoid any authentication errors, before the current token expires, you must repeat the steps outlined above to generate a new access token. If a request is made with an expired token and/or an invalid token, the Bulk Export API will generate a `401` HTTP status code indicating that it's an unauthorized request.

8. Now that you have your token, you're ready to start using the Bulk Export API to export your data.

2. 2 | Create a Job to Export Contact Data

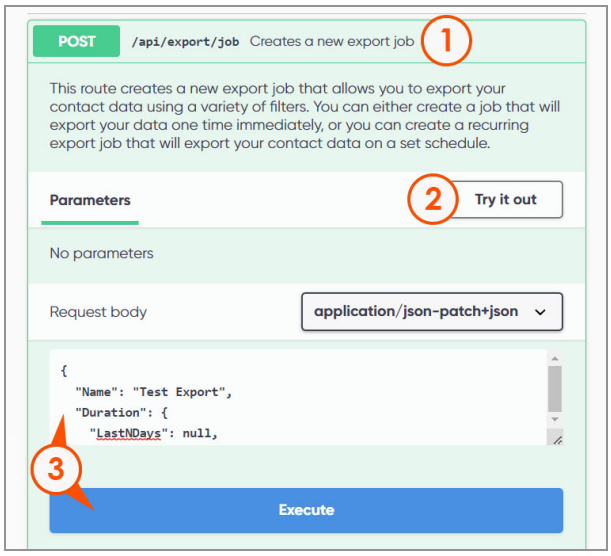
The Bulk Export API allows you to export a variety of contact data to CSV (along with a JSON-formatted schema file for each data type that contains further information about the export) through one easy-to-use route. With this route, you can:

- Export contact data, including transcripts, categories, scores, and much more. (See Export Data Types for a full list of the types of data that can be exported.)
- Export your chosen data one time immediately, or you can create a recurring job that will export and deliver your data on a continuous schedule.

The route gives you the variety and flexibility you need to export the data you're interested in on the schedule that works for you.

To create a job to export contact data, choose the option below you'd like to use and follow the steps outlined there:

Using the Interactive Documentation



1. Press the **POST /api/export/job** route to expand it.
2. Press **Try it out**.
3. Enter the following into the request body, edit the payload values, and press **Execute**:

Using a Third-Party Application

1. Create a `POST` request with the `/api/export/job` route and the CallMiner Bulk Export API URL.
2. Set the following request headers:
   - **Authorization**: Set the value to the word *Bearer* followed by a space and then the authorization token. For example, *Bearer abc1234*.
   - **Content-Type**: Set the value to *application/json; charset=utf-8*.

   For example:

   ```
   POST /api/export/job
   Host: Bulk_Export_API_URL
   Authorization: Bearer abc1234
   Content-Type: application/json;
   charset=utf-8
   ```

3. Enter the following into the request body, edit the payload values, and send the request:

Here's an example of the request you might create:

```
{
   "Name": "Example Export",
   "Duration": {
      "LastNDays": null,
      "LastNHours": null,
      "TimeFrame": "LastWeek",
      "StartDate": null,
      "EndDate": null,
      "SearchMode": "ClientCaptureDate"
   },
   "DataTypes": [
      "Contacts",
      "Transcripts"
   ],
   "SearchFilters": [
        {
          "CategoryId": null,
          "TagId": null,
          "SectionId": null,
          "ScoreId": null,
          "AlertId": null,
          "TableFilterTypes": null,
          "ColumnName": "Agent",
          "Operator": "=",
          "Operand1": "John Doe",
          "Operand2": null
        }
      ],
   "NotificationMethod": "Email",
   "EmailRecipients": [
      "tinker.bell@callminer.com",
      "john.darling@callminer.com"
   ],
   "WebhookId": null,
   "StorageTargetName": "Example Storage",
   "Schedule": "0 30 8 ? * 4#1"
}
```

- **Name** (string): ***Required***. The name of the export job.
- **Duration** (object): ***Required***. The timeframe that data should be exported from when the job runs.

  Choose from the following options:

Option	How to Set It	Example

#1: Export my data from the *X* number of previous days	Set `LastNDays` to the number of previous days that the data should be exported from every time the job runs. Example: enter *7* to export data from the last seven days every time.	`"Duration": {`
		`"LastNDays": 7,`
		`"LastNHours": null,`
		`"TimeFrame": null,`
		`"StartDate": null,`
		`"EndDate": null`
		`},`

#2 Export my data from the *X* number of previous hours	Set `LastNHours` to the number of previous hours that the data should be exported from every time the job runs. Example: enter *1* to export data	`"Duration": {`
		`"LastNDays": null,`
		`"LastNHours": 1,`
		`"TimeFrame": null,`
		`"StartDate": null,`
		`"EndDate": null`
		`},`

Option	How to Set It	Example
	from the previous hour every time.	

#3: Export my data from a predefined timeframe	Set **`TimeFrame`** to either *Yesterday*, *LastWeek*, *ThisMonth*, or *LastMonth*.	`"Duration": {`
		`"LastNDays": null,`
		`"LastNHours": null,`
		`"TimeFrame": "LastWeek",`
		`"StartDate": null,`
		`"EndDate": null`
		`},`

#4: Export my data from a custom date range
 	
Important You may only export data in full-day increments with this option—for example, 1 full day, 2 full days, etc.
	Set **`TimeFrame`** to *Custom*, and enter a date (in ISO 8601 format in UTC) for **`StartDate`** and **`EndDate`**.	`"Duration": {`
		`"LastNDays": null,`
		`"LastNHours": null,`
		`"TimeFrame": "Custom",`
		`"StartDate": "2024-10-07T12:01:00.000Z",`
		`"EndDate": "2024-10-08T12:01:00.000Z"`
		`},`

Additionally, use the **`SearchMode`** field to modify what the time period or **`Duration`** applies to. Example:

```
"Duration": {
```

```
        "LastNDays": 7,
        "LastNHours": null,
        "TimeFrame": null,
        "StartDate": null,
        "EndDate": null,
        "SearchMode": "ClientCaptureDate"
    }
```

Choose from the following values:

Option	Enter This Value
#1 The system will pull back all contacts whose `ClientCaptureDate` falls within the provided `Duration`. **Note**: This is the default option. If you don't include `SearchMode` in the request, the export will use this as the default.	*ClientCaptureDate*
#2 The system will pull back all contacts whose `CreateDate` falls within the provided `Duration`.	*CreateDate*
#3 The system will pull back all contacts that have been modified by a user or retroprocessed by the system within the provided `Duration`. **Note**: This option replaces the deprecated `IncludeUpdatedContacts` field.	*Updated*
#4 The system will pull back all contacts that have been modified by a user or retroprocessed by the system, as well as all new contacts in the system within the provided `Duration`. **Note**: We recommend using this option if you set `LastNHours` to *4* (or fewer) hours as your export `Duration` to ensure no contacts are missed and your export includes all contacts that have been recently processed by the system.	*NewAndUpdated*

- **DataTypes** (array of strings): ***Required***. The list of data types that will be exported. Choose one or more of the following data types: *Contacts*, *Categories*, *Category_components*, *Scores*, *Score_indicators*, *Tags*, *Alerts*, *Comments*, *Events*, *Coach_insights*, *Coach_workflow*, *Transcripts_by_speaker*, *Ai_*

summaries, *Client_Ids*, *Email_metadata*, and *Audio*.

> **Important**: To use the `Audio` data type, you must enter a valid `StorageTargetName` and have your exported files delivered to a third-party storage target. You cannot export audio files to email or webhook.

For more information on what each data type includes, see Export Data Types.

- **MetadataFilters** (object): ***Deprecated***. Use the `SearchFilters` array below instead to filter your export by metadata.
- **SearchFilters** (array): Used to filter the data returned in the export. This is what the base `SearchFilters` array looks like:

```
"SearchFilters": [
        {
          "CategoryId": null,
          "TagId": null,
          "SectionId": null,
          "ScoreId": null,
          "AlertId": null,
          "TableFilterTypes": null,
          "ColumnName": "Agent",
          "Operator": "=",
          "Operand1": "John Doe",
          "Operand2": null
        }
      ]
```

To set it, choose from the following filter options:

Option	How to Set It	Example

#1 Filter your export by Category ID (s).	Enter each `CategoryId` value you want to filter by into its own object within the `SearchFilters` array. (To obtain a list of `CategoryId` values in your Eureka system, use the	`"SearchFilters": [`
		`{`
		`"CategoryId": 12345,`
		`"Operator": "="`
		`},`
		`{`

Option	How to Set It	Example
	Retrieve Content SearchFilters Values route.) Then, set **Operator** to = (equals).	7 "CategoryId": 67890, 8 "Operator": "=" 9 } 10]

#2 Filter your export by Tag ID(s).	Enter each **TagId** value you want to filter by into its own object within the **SearchFilters** array. (To obtain a list of **TagId** values in your Eureka system, use the Retrieve Content SearchFilters Values route.)
	
	Then, set **Operator** to = (equals).
		1 "SearchFilters": [
		2 {
		3 "TagId": 573621,
		4 "Operator": "="
		5 },
		6 {
		7 "TagId": 48472,
		8 "Operator": "="
		9 }
		10]

#3 Filter your export by Section ID(s).	Enter each **SectionId** value you want to filter by into its own object within the **SearchFilters** array. (To obtain a list of **SectionId** values in your Eureka system, use the Retrieve Content SearchFilters Values route.)
	
	Then, set **Operator** to = (equals).
		1 "SearchFilters": [
		2 {
		3 "SectionId": 12345,
		4 "Operator": "="
		5 },
		6 {
		7 "SectionId": 67890,
		8 "Operator": "="
		9 }
		10]

#4 Filter your export by Alert ID(s).	Enter each **AlertId** value you want to filter by into its own object within the **SearchFilters** array. (To obtain a list of **AlertId**	1 "SearchFilters": [2 { 3 "AlertId": 12345,

Option	How to Set It	Example

	values in your Eureka system, use the Retrieve Content SearchFilters Values route.)
	
	Then, set **`Operator`** to = (equals).
		`"Operator": "="`
		`},`
		`{`
		`"AlertId": 67890,`
		`"Operator": "="`
		`}`
		`]`

#5 Filter your export by contacts that fell into a specific score range based on the name of the range.	Enter the **`ScoreId`** value of the score the range belongs to in an object in the **`SearchFilters`** array. Then, set **`Operator`** to = (equals) and **`Operand1`** to the friendly name of the score range. (To obtain a list of **`ScoreId`** and score **`RangeFriendlyName`** values in your Eureka system, use the Retrieve Content SearchFilters Values route.)	**Example**: You want to export every contact that fell into the *Poor* range of your **Politeness** score.
		
		`"SearchFilters": [`
		`{`
		`"ScoreId": 573621,`
		`"Operator": "=",`
		`"Operand1": "Poor"`
		`}`
		`]`

#6 Filter your export by contacts that fell into a specific score range based on the actual range values.	Enter the **`ScoreId`** value of the score the range belongs to in an object in the **`SearchFilters`** array. Then, set **`Operator`** to *between*, **`Operand1`** to the lower value, and **`Operand2`** to the higher value you are looking for. (To obtain a list of **`ScoreId`** and score **`RangeMin`** and **`RangeMax`** values in your Eureka	**Example**: You want to export every contact that scored anywhere from *0* to *50* on your **Politeness** score.
		
		`"SearchFilters": [`
		`{`
		`"ScoreId": 573621,`
		`"Operator": "between",`
		`"Operand1": "0",`
		`"Operand2": "50"`
		`}`

Option	How to Set It	Example
	system, use the Retrieve Content SearchFilters Values route.)	`]`

#7 Filter your export by one or more consecutive words within the comments on each contact.	Set the **TableFilterTypes** field to *CallComment* within the **SearchFilters** array. Then, set the **Operator** field to either *beginswith*, *contains*, *endswith*, or = (equals)and the **Operand1** value to the word(s) within the comments you want to filter by.	**Example**: You want to pull back every contact that had a comment associated with it with the words "review later".
		`"SearchFilters": [`
		`{`
		`"TableFilterTypes": "CallComment",`
		`"Operator": "=",`
		`"Operand1": "review later"`
		`}`
		`]`

#8 Filter your export by one or more consecutive words within the comments on each score associated with various contacts.	Set the **TableFilterTypes** field to *ScoreComment* within the **SearchFilters** array. Then, set the **Operator** field to either *beginswith*, *contains*, *endswith*, or = (equals)and the **Operand1** value to the word(s) within the comments you want to filter by.	**Example**: You want to pull back every contact that had a comment on a score with the words "needs updated".
		`"SearchFilters": [`
		`{`
		`"TableFilterTypes": "ScoreComment",`
		`"Operator": "=",`
		`"Operand1": "needs updated"`
		`}`
		`]`

#9 Filter your export by the message date of new	Set the **TableFilterTypes** field to *SegmentStartDay* within the **SearchFilters** array. Then, set the	**Example**: You want to pull back every text-based contact that had a new message added to the text contact thread any day from November 1st through November 3rd.

Option	How to Set It	Example

threads in a text-based contact.	**Operator** field to = (equals), > (greater than), or < (less than), and the **Operand1** value to a date string.
	
	Or, use the *between* operator and provide a date value for both **Operand1** and **Operand2**.
		`"SearchFilters": [`
		`  {`
		`  "TableFilterTypes": "SegmentStartDay",`
		`  "Operator": "between",`
		`  "Operand1": "11/01/2024",`
		`  "Operand2": "11/03/2024"`
		`  }`
		`]`

#10 Filter your export by included metadata.	Enter each metadata item you want to filter by into its own object within the **SearchFilters** array.
	
	Then, set the **ColumnName** values to the **ColumnName** of the metadata items you want to filter by. (To obtain a list of **ColumnName** values in your Eureka system, use the Retrieve Metadata Information route.)
	
	Set **Operator** to =(equals) and **Operand1** to the metadata value you want to filter by.
		Example: You want your export to only include contacts handled by agent *John Smith* and agent *Jane Doe*.
		
		`"SearchFilters": [`
		`  {`
		`  "ColumnName": "Agent",`
		`  "Operator": "=",`
		`  "Operand1": "John Smith"`
		`  },`
		`  {`
		`  "ColumnName": "Agent",`
		`  "Operator": "=",`
		`  "Operand1": "Jane Doe"`
		`  }`
		`  ]`

#11 Filter your export by excluded metadata.	Enter each metadata item you want to filter by into its own object within the **SearchFilters** array.
	
	Then, set the **ColumnName** values to the **ColumnName** of the metadata items you want to filter by. (To obtain
		Example: You want your export to include all contacts *except* those handled by agent *John Smith* or agent *Jane Doe*.
		
		`"SearchFilters": [`
		`  {`

Option	How to Set It	Example

	a list of **ColumnName** values in your Eureka system, use the Retrieve Metadata Information route.)
	
	Set **Operator** to *!=* (not equals) and **Operand1** to the metadata value you want to filter by.
		`"ColumnName": "Agent",`
		`"Operator": "!=",`
		`"Operand1": "John Smith"`
		`},`
		`{`
		`"ColumnName": "Agent",`
		`"Operator": "!=",`
		`"Operand1": "Jane Doe"`
		`}`
		`]`

#12 Filter your export by contacts that fell into a specific metadata range based on the name of the range.	Enter the **ColumnName** value of the metadata item the range belongs to in the **SearchFilters** array. Then, set **Operator** to = (equals) and **Operand1** to the name of the metadata range. (To obtain a list of **ColumnName** values and metadata **Ranges** name values in your Eureka system, use the Retrieve Metadata Information route.)	**Example**: You want to export every contact that fell into the *Longest Hold Time* range of your **Hold Time** metadata item.
		
		`"SearchFilters": [`
		`{`
		`"ColumnName": "Hold Time",`
		`"Operator": "=",`
		`"Operand1": "Longest Hold Time"`
		`}`
		`]`

#13 Export data from a specific time range from the previous day.	Within the **SearchFilters** array, enter *ClientCaptureTime* as the **ColumnName** value and set **Operator** to *between*. Then, use *HH:mm:ss* format and enter the start time of the range you want to export as the value for **Operand1** and the end	**Example**: You want to export every contact that occurred between 9 AM and 5 PM the day prior.
		
		`"Duration": {`
		`"LastNDays": 1,`
		`"LastNHours": null,`
		`"TimeFrame": null,`

<table>
<tr><th>Option</th><th>How to Set It</th><th>Example</th></tr>
<tr><td></td><td>time of the range as the value for Operand2.

Set your Duration object to LastNDays with a value of 1.</td><td><pre>
5 "StartDate": null,
6 "EndDate": null
7 },
8 "SearchFilters": [
9 {
10 "ColumnName": "ClientCaptureTime",
11 "Operator": "between",
12 "Operand1": "09:00:00",
13 "Operand2": "17:00:00"
14 }
15]
</pre></td></tr>
<tr><td>#14 Filter your export by other metadata options.</td><td>The sky (and your system configuration) is the limit here. Get creative! You can use many different combinations of metadata filters depending on your specific use-case and business needs. You can also use a combination of the following Operator values with one or both Operand fields to

achieve your desired results:

<table><tr><th>Operator Value</th><th>Operator Meaning</th></tr><tr><td>=</td><td>equals</td></tr><tr><td>!=</td><td>does not equal</td></tr></table></td><td>Example: You want to filter your export by audio contacts that had Long Hold Time and were handled by your Customer Service department.

<pre>
1 "SearchFilters": [
2 {
3 "ColumnName": "Contact Type",
4 "Operator": =,
5 "Operand1": "call"
6 {
7 "ColumnName": "Department",
8 "Operator": "=",
9 "Operand1": "Customer Service"
10 },
11 {
12 "ColumnName": "Hold Time",
13 "Operator": "=",
14 "Operand1": "Long Hold Time"
15 }
16]
</pre></td></tr>
</table>

Option	How to Set It	Example

Operator Value	Operator Meaning
>	is greater than
>=	is greater than or equal to
!>	is not greater than
<	is less than
<=	is less than or equal to
!<	is not less than
contains	the value contains
!contains	the value does not contain
between	the value is between
!between	the value is not between

<table>
<tr><th>Option</th><th>How to Set It</th><th>Example</th></tr>
<tr><td></td><td>
<table>
<tr><th>Operator Value</th><th>Operator Meaning</th></tr>
<tr><td>beginswith</td><td>the value begins with</td></tr>
<tr><td>!beginswith</td><td>the value does not begin with</td></tr>
<tr><td>endswith</td><td>the value ends with</td></tr>
<tr><td>!endswith</td><td>the value does not end with</td></tr>
<tr><td>isnull</td><td>the value is null</td></tr>
<tr><td>!isnull</td><td>the value is not null</td></tr>
</table>
See SearchFilters Basics for more details and examples.
</td><td></td></tr>
</table>

Option	How to Set It	Example

#15 Combine multiple `SearchFilters` options and use them together.	Enter as many objects within the `SearchFilters` array as you'd like to achieve the exact filtered results you are looking for.
	
	If you do not wish to use any of the other `SearchFilters` options within a given object, either remove the fields from the request or set them to *null*.
		Example: You want to filter your export by contacts that had a hit on the category with ID *126336* and were handled by agent *John Doe*.
		
		`"SearchFilters": [`
		`{`
		`"CategoryId": 126336,`
		`"Operator": "="`
		`},`
		`{`
		`"ColumnName": "Agent",`
		`"Operator": "=",`
		`"Operand1": "John Doe"`
		`}`
		`]`

- **IncludeUpdatedContacts** (boolean): ***Deprecated***. Set the `SearchMode` field in the `Duration` object to *Updated* instead to achieve the same result.
- **NotificationMethod** (string): The method that will be used to deliver the exported data. Or, if you've chosen to use a third-party storage target, this will be the method used to send you notifications relating to your export job. Choose either *Email* or *Webhook*.

> **Important**: Before choosing *Webhook* as your `NotificationMethod`, your webhook must be fully connected to the CallMiner system and the connection must be validated. If the webhook hasn't been validated, you will not receive your requested data and notifications from the Bulk Export API. See the "Connect Your Webhook to the CallMiner System" section of the *CallMiner Webhook Management API User Guide* for more information.

- **EmailRecipients** (array of strings): Required if `NotificationMethod` is set to *Email*. The list of one or more email addresses that will receive notifications from the export job.
- **WebhookId** (string): Required if `NotificationMethod` is set to *Webhook*. The unique identifier of the webhook that will receive notifications from the export job.

- **StorageTargetName** (string): The `Name` of the storage target where your exported data should be delivered, if applicable.

  > **Important**: Before choosing a storage target as your delivery location, you must first add the storage target you want to use to the Bulk Export API. See Create or Update a Third-Party Storage Target for more information.

- **Schedule** (string): The schedule used to export the data. You have two options for how and when the data will be exported:
  - Have the data exported one time, immediately. To do so, remove the `Schedule` field from the request entirely.
  - Have the data exported multiple times on a set delivery schedule. To do so, enter a Quartz cron expression that defines the schedule you want the data to be exported on. Please note that any times you enter for your schedule should be in UTC and that the most frequent delivery schedule you can set is exporting your data every 1 hour.

    > **Pro-tip**: See Quartz Cron Basics for more information or use an online Quartz cron expression builder to easily create your `Schedule` value.

The Response

> **Important**: If an error occurs, an HTTP status code in the `400` or `500` range is returned. Refer to the message provided with the error and HTTP Status Codes to troubleshoot what went wrong, correct the request, and try again.

After you send the request, if the request is successful, you will receive a `201` HTTP status code and a response body that contains all of the information you sent in with the request, as well as the following fields:

- **Id** (string): The unique identifier of the export job.
- **TenantApiKey** (string): The Tenant API Key for the tenant associated with the export job.
- **CreatedBy** (string): The username of the user who created the export job.
- **CreateDate** (datetime): The date and time when the export job was created.

And you're all set! Depending on which `NotificationMethod` you chose, you'll either receive an email or a webhook event with a link where you can download your exported data (or the file path where your data was delivered, if using a third-party storage target).

The file path or download link will provide a main zip file, which contains the JSON-formatted schema file(s) for each requested data type. (For an example of what information this schema file contains, see JSON-Formatted Schema File Contents.) Additionally, the main zip file will contain one Gzipped CSV file for each data type you requested; these Gzipped CSV files are where you will find all of the contact data you wanted to export.

If you chose to omit the `Schedule` field, your data will be delivered one time immediately. If you chose a continuous delivery schedule, your data will be delivered according to the Quartz cron schedule you provided in the request.

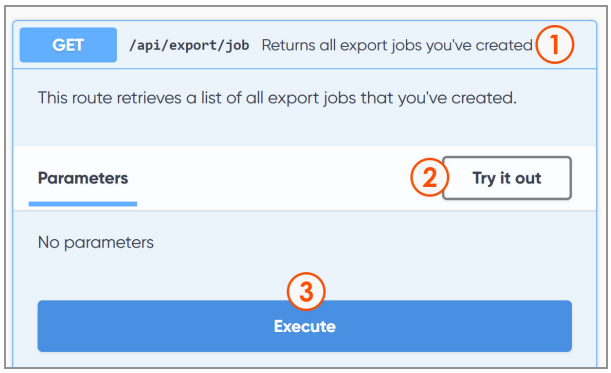
> **Note**: If you want to import your contact data into an integration platform, instead of waiting for the email or webhook event that contains the link where you can download your data, you can set up your application to poll our **GET /api/export/history** route. Have your application take the `Id` of the export job you received in this response and enter it into the `Id` parameter on the **GET /api/export/history** route. Continue polling the history route with the `Id` value until the `Status` for the export changes to *Completed* and the `DownloadUrl` and `DownloadEndpoint` values are populated.
>
> > **Pro-tip**: We recommend having your application poll our **GET /api/export/history** route at 60 second intervals. Due to the large amount of data in some exports, it may take a few minutes for the data to be ready for download into your application.
>
> Once the `DownloadUrl` and `DownloadEndpoint` values appear, the data is ready and it can be imported into your integration platform. For more information on using the **GET /api/export/history** route, see Retrieve the History of Data Exports for All Export Jobs.

> **Note**: If, at any point, you decide that you'd like to update or delete one of the scheduled export jobs you've created, see Update a Recurring Export Job and Delete One of Your Export Jobs.

2. 3 | Retrieve a List of All of Your Export Jobs

To retrieve a list of all of your export jobs, choose the option below you'd like to use and follow the steps outlined there:

Using the Interactive Documentation



1. Press the **GET /api/export/job** route to expand it.
2. Press **Try it out**.
3. Press **Execute**.

Using a Third-Party Application

1. Create a `GET` request with the `/api/export/job` route and the CallMiner Bulk Export API URL.
2. Set the following request headers:
   - **Authorization**: Set the value to the word *Bearer* followed by a space and then the authorization token. For example, *Bearer abc1234*.
   - **Content-Type**: Set the value to *application/json; charset=utf-8*.

   For example:

   ```
   GET /api/export/job
   Host: Bulk_Export_API_URL
   Authorization: Bearer abc1234
   Content-Type: application/json;
   charset=utf-8
   ```
3. Send the request.

The Response

> **Important**: If an error occurs, an HTTP status code in the `400` or `500` range is returned. Refer to the message provided with the error and HTTP Status Codes to troubleshoot what went wrong, correct the request, and try again.

If the request is successful, you will receive a `200` HTTP status code and a response body that contains all of the settings for each export job. The response fields you may be interested in are:

- **Id** (string): The unique identifier of the export job.
- **Duration** (object): The timeframe that data should be exported from when the job runs, along with the `SearchMode` used to export the data.
- **DataTypes** (array of strings): The list of data types that will be exported. Refer to Export Data Types for more information.
- **SearchFilters** (array): The metadata and content filters used to filter the data returned in the export.
- **NotificationMethod** (string): The method that will be used to deliver the exported data. Or, if you've chosen to use a third-party storage target, this will be the method used to send you notifications relating to your export job.Can be either *Email* or *Webhook*.
- **StorageTargetName** (string): The `Name` of the storage target where your exported data should be delivered, if applicable.
- **Schedule** (string): The Quartz cron schedule used to export the data.

For a reminder of what the other fields in the response mean, check out the Bulk Export API interactive documentation.

> **Note**: If you'd like to update or delete a job that was returned in the list, note the `Id` value for the job returned in this route. This `Id` value is what you will enter into the `id` parameter of the Update a Recurring Export Job route or the Delete One of Your Export Jobs route.
>
> You may also want to copy the object for the job from this response to paste it into the request of the Update a Recurring Export Job route to make updating the job easier.

2. 4 | Retrieve a Specific Export Job

To retrieve a specific export job, choose the option below you'd like to use and follow the steps outlined there:

Using the Interactive Documentation

1. Press the **GET /api/export/job/{id}** route to expand it.
2. Press **Try it out**.
3. Enter the ID of the export job you want to retrieve into the **id** parameter.
4. Press **Execute**.

Using a Third-Party Application

1. Create a `GET` request with the `/api/export/job/{id}` route, the CallMiner Bulk Export API URL, and the ID of the export job you want to retrieve. For example: *GET /api/export/job/5678* (where *5678* is the ID of the job).
2. Set the following request headers:
   - **Authorization**: Set the value to the word *Bearer* followed by a space and then the authorization token. For example, *Bearer abc1234*.
   - **Content-Type**: Set the value to *application/json; charset=utf-8*.

   For example:

   ```
   GET /api/export/job/5678
   Host: Bulk_Export_API_URL
   Authorization: Bearer abc1234
   Content-Type: application/json;
   charset=utf-8
   ```
3. Send the request.

The Response

Important: If an error occurs, an HTTP status code in the `400` or `500` range is returned. Refer to the message provided with the error and HTTP Status Codes to troubleshoot what went wrong, correct the request, and try again.

If the request is successful, you will receive a `200` HTTP status code and a response body that contains all of the settings for the export job. The response fields you may be interested in are:

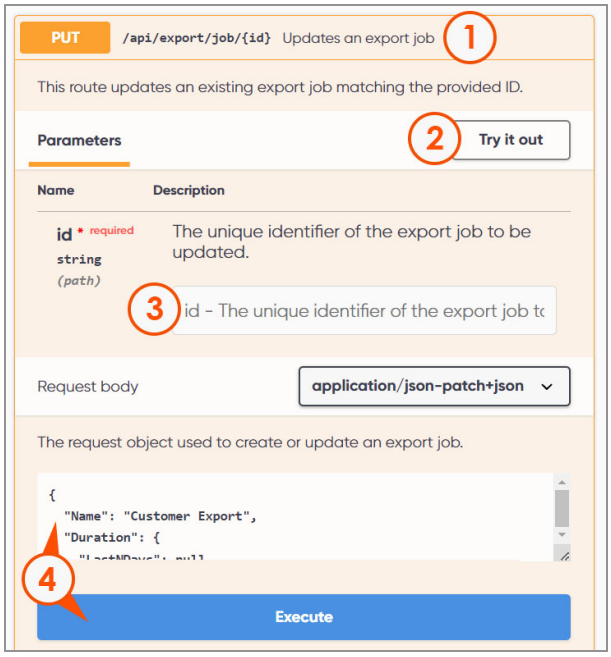
- **Id** (string): The unique identifier of the export job.
- **Duration** (object): The timeframe that data should be exported from when the job runs, along with the `SearchMode` used to export the data.
- **DataTypes** (array of strings): The list of data types that will be exported. Refer to Export Data Types for more information.
- **SearchFilters** (array): The metadata and content filters used to filter the data returned in the export.
- **NotificationMethod** (string): The method that will be used to deliver the exported data. Or, if you've chosen to use a third-party storage target, this will be the method used to send you notifications relating to your export job.Can be either *Email* or *Webhook*.
- **StorageTargetName** (string): The `Name` of the storage target where your exported data should be delivered, if applicable.
- **Schedule** (string): The Quartz cron schedule used to export the data.

For a reminder of what the other fields in the response mean, check out the Bulk Export API interactive documentation.

2. 5 | Update a Recurring Export Job

Please note that you can only update recurring export jobs. If you omitted the cron `Schedule` when you created a job and set it to run one time, immediately upon creation, you cannot update the job after the fact. To update an export job, choose the option below you'd like to use and follow the steps outlined there:

Using the Interactive Documentation



1. Press the **PUT /api/export/job/{id}** route.
2. Press **Try it out**.
3. Enter the ID of the export job you want to update into the **id** parameter.

   > **Note**: Use the Retrieve a List of All of Your Export Jobs route to find the `id` value for the job you want to update.

4. Enter the following into the request body, edit the payload values, and press **Execute**:

Using a Third-Party Application

1. Create a `PUT` request with the `/api/export/job/{id}` route, the CallMiner Bulk Export API URL, and the ID of the export job you want to update. For example: *PUT /api/export/job/5678* (where *5678* is the ID of the job).

   > **Note**: Use the Retrieve a List of All of Your Export Jobs route to find the `id` value for the job you want to update.

2. Set the following request headers:
   - **Authorization**: Set the value to the word *Bearer* followed by a space and then the authorization token. For example, *Bearer abc1234*.
   - **Content-Type**: Set the value to *application/json; charset=utf-8*.

   For example:

   ```
   PUT /api/export/job/5678
   Host: Bulk_Export_API_URL
   Authorization: Bearer abc1234
   Content-Type: application/json;
   charset=utf-8
   ```

3. Enter the following into the request body, edit the payload values, and send the request:

> **Pro-tip**: To avoid having to re-enter all of the request values in this route, simply copy the object for the job from the Retrieve a List of All of Your Export Jobs response and paste it into this request. Then, all you need to do is modify the values you want to change before sending the request.

Here's an example of the request you might create:

```
{
   "Name": "Example Export",
   "Duration": {
      "LastNDays": null,
      "LastNHours": null,
      "TimeFrame": "LastWeek",
      "StartDate": null,
      "EndDate": null,
      "SearchMode": "ClientCaptureDate"
   },
   "DataTypes": [
      "Contacts",
      "Transcripts"
   ],
   "SearchFilters": [
        {
          "CategoryId": null,
          "TagId": null,
          "SectionId": null,
          "ScoreId": null,
          "AlertId": null,
          "TableFilterTypes": null,
          "ColumnName": "Agent",
          "Operator": "=",
          "Operand1": "John Doe",
          "Operand2": null
        }
      ],
   "NotificationMethod": "Email",
   "EmailRecipients": [
      "tinker.bell@callminer.com",
      "john.darling@callminer.com"
   ],
   "WebhookId": null,
```

```
    "StorageTargetName": "Example Storage",
    "Schedule": "0 30 8 ? * 4#1"
}
```

- **Name** (string): ***Required***. The name of the export job.
- **Duration** (object): ***Required***. The timeframe that data should be exported from when the job runs.

  Choose from the following options:

Option	How to Set It	Example

#1: Export my data from the *X* number of previous days	Set **`LastNDays`** to the number of previous days that the data should be exported from every time the job runs. Example: enter *7* to export data from the last seven days every time.	`"Duration": {`
		`"LastNDays": 7,`
		`"LastNHours": null,`
		`"TimeFrame": null,`
		`"StartDate": null,`
		`"EndDate": null`
		`},`

#2 Export my data from the *X* number of previous hours	Set **`LastNHours`** to the number of previous hours that the data should be exported from every	`"Duration": {`
		`"LastNDays": null,`
		`"LastNHours": 1,`
		`"TimeFrame": null,`
		`"StartDate": null,`
		`"EndDate": null`
		`},`

Option	How to Set It	Example
	time the job runs. Example: enter *1* to export data from the previous hour every time.	

#3: Export my data from a predefined timeframe	Set **`TimeFrame`** to either *Yesterday*, *LastWeek*, *ThisMonth*, or *LastMonth*.	`"Duration": {`
		`"LastNDays": null,`
		`"LastNHours": null,`
		`"TimeFrame": "LastWeek",`
		`"StartDate": null,`
		`"EndDate": null`
		`},`

#4: Export my data from a custom date range
 	
Important You may only export data in full-day increments with this option—for example, 1 full day, 2 full days, etc.
	Set **`TimeFrame`** to *Custom*, and enter a date (in ISO 8601 format in UTC) for **`StartDate`** and **`EndDate`**.	`"Duration": {`
		`"LastNDays": null,`
		`"LastNHours": null,`
		`"TimeFrame": "Custom",`
		`"StartDate": "2024-10-07T12:01:00.000Z",`
		`"EndDate": "2024-10-08T12:01:00.000Z"`
		`},`

Additionally, use the **SearchMode** field to modify what the time period or **Duration** applies to. Example:

```
"Duration": {
      "LastNDays": 7,
      "LastNHours": null,
      "TimeFrame": null,
      "StartDate": null,
      "EndDate": null,
      "SearchMode": "ClientCaptureDate"
   }
```

Choose from the following values:

Option	Enter This Value
#1 The system will pull back all contacts whose **ClientCaptureDate** falls within the provided **Duration**. **Note**: This is the default option. If you don't include **SearchMode** in the request, the export will use this as the default.	*ClientCaptureDate*
#2 The system will pull back all contacts whose **CreateDate** falls within the provided **Duration**.	*CreateDate*
#3 The system will pull back all contacts that have been modified by a user or retroprocessed by the system within the provided **Duration**. **Note**: This option replaces the deprecated **IncludeUpdatedContacts** field.	*Updated*
#4 The system will pull back all contacts that have been modified by a user or retroprocessed by the system, as well as all new contacts in the system within the provided **Duration**. **Note**: We recommend using this option if you set **LastNHours** to *4* (or fewer) hours as your export **Duration** to ensure no contacts are missed and your export includes all contacts that have been recently processed by the system.	*NewAndUpdated*

- **DataTypes** (array of strings): ***Required***. The list of data types that will be exported. Choose one or more of the following data types: *Contacts*, *Categories*, *Category_components*, *Scores*, *Score_indicators*, *Tags*, *Alerts*, *Comments*, *Events*, *Coach_insights*, *Coach_workflow*, *Transcripts_by_speaker*, *Ai_summaries*, *Client_Ids*, *Email_metadata*, and *Audio*.

  > **Important**: To use the `Audio` data type, you must enter a valid `StorageTargetName` and have your exported files delivered to a third-party storage target. You cannot export audio files to email or webhook.

  For more information on what each data type includes, see Export Data Types.

- **MetadataFilters** (object): ***Deprecated***. Use the `SearchFilters` array below instead to filter your export by metadata.

- **SearchFilters** (array): Used to filter the data returned in the export. This is what the base `SearchFilters` array looks like:

```
"SearchFilters": [
        {
          "CategoryId": null,
          "TagId": null,
          "SectionId": null,
          "ScoreId": null,
          "AlertId": null,
          "TableFilterTypes": null,
          "ColumnName": "Agent",
          "Operator": "=",
          "Operand1": "John Doe",
          "Operand2": null
        }
      ]
```

  For a reminder of the many filter options available to you, see SearchFilters Basics.

- **IncludeUpdatedContacts** (boolean): ***Deprecated***. Set the `SearchMode` field in the `Duration` object to *Updated* instead to achieve the same result.

- **NotificationMethod** (string): The method that will be used to deliver the exported data. Or, if you've chosen to use a third-party storage target, this will be the method used to send you notifications relating to your export job. Choose either *Email* or *Webhook*.

> **Important**: Before choosing *Webhook* as your `NotificationMethod`, your webhook must be fully connected to the CallMiner system and the connection must be validated. If the webhook hasn't been validated, you will not receive your requested data and notifications from the Bulk Export API. See the "Connect Your Webhook to the CallMiner System" section of the *CallMiner Webhook Management API User Guide* for more information.

- **EmailRecipients** (array of strings): Required if `NotificationMethod` is set to *Email*. The list of one or more email addresses that will receive notifications from the export job.
- **WebhookId** (string): Required if `NotificationMethod` is set to *Webhook*. The unique identifier of the webhook that will receive notifications from the export job.
- **StorageTargetName** (string): The `Name` of the storage target where your exported data should be delivered, if applicable.

> **Important**: Before choosing a storage target as your delivery location, you must first add the storage target you want to use to the Bulk Export API. See Create or Update a Third-Party Storage Target for more information.

- **Schedule** (string): The schedule used to export the data. You have two options for how and when the data will be exported:
  - Have the data exported one time, immediately. To do so, remove the `Schedule` field from the request entirely.
  - Have the data exported multiple times on a set delivery schedule. To do so, enter a Quartz cron expression that defines the schedule you want the data to be exported on. Please note that any times you enter for your schedule should be in UTC and that the most frequent delivery schedule you can set is exporting your data every 1 hour.

    > **Pro-tip**: See Quartz Cron Basics for more information or use an online Quartz cron expression builder to easily create your `Schedule` value.

The Response

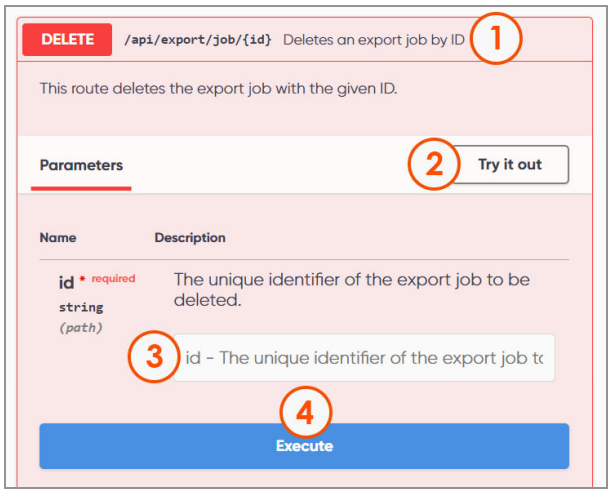
> **Important**: If an error occurs, an HTTP status code in the `400` or `500` range is returned. Refer to the message provided with the error and HTTP Status Codes to troubleshoot what went wrong, correct the request, and try again.

If the request is successful, you will receive a `201` HTTP status code and a response body that contains all of the information you sent in with the request. You have successfully updated your export job. Your data will be exported and delivered according to the settings you updated in the request.

2. 6 | Delete One of Your Export Jobs

To delete one of your export jobs, choose the option below you'd like to use and follow the steps outlined there:

Using the Interactive Documentation



1. Press the **DELETE /api/export/job/{id}** route to expand it.
2. Press **Try it out**.
3. Enter the ID of the export job you want to delete into the **id** parameter.

   > **Note**: Use the Retrieve a List of All of Your Export Jobs route to find the `id` value for the job you want to delete.

4. Press **Execute**.

Using a Third-Party Application

1. Create a `DELETE` request with the `/api/export/job/{id}` route, the CallMiner Bulk Export API URL, and the ID of the export job you want to delete. For example: *DELETE /api/export/job/5678* (where *5678* is the ID of the job).

   > **Note**: Use the Retrieve a List of All of Your Export Jobs route to find the `id` value for the job you want to delete.

2. Set the following request headers:
   - **Authorization**: Set the value to the word *Bearer* followed by a space and then the authorization token. For example, *Bearer abc1234*.
   - **Content-Type**: Set the value to *application/json; charset=utf-8*.

   For example:

   ```
   DELETE /api/export/job/5678
   Host: Bulk_Export_API_URL
   Authorization: Bearer abc1234
   Content-Type: application/json;
   charset=utf-8
   ```

3. Send the request.

The Response

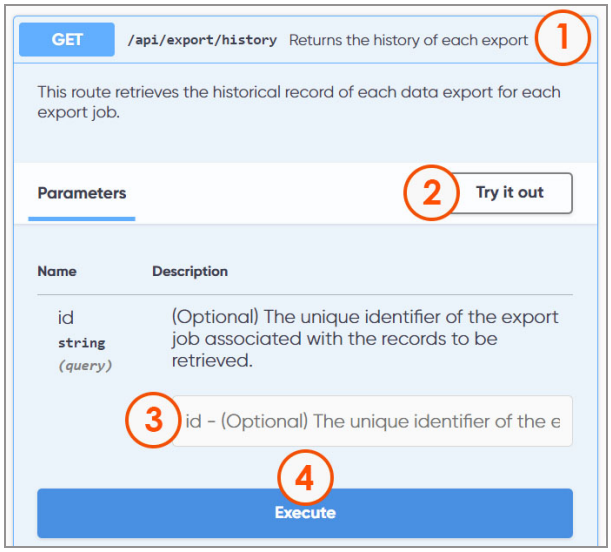
> **Important**: If an error occurs, an HTTP status code in the 400 or 500 range is returned. Refer to the message provided with the error and HTTP Status Codes to troubleshoot what went wrong, correct the request, and try again.

If the request is successful, you will receive a 200 HTTP status code. You have successfully deleted the export job. It will no longer run or exist in the system.

2. 7 | Retrieve the History of Data Exports for All Export Jobs

A history record is generated every time a job exports data; it includes the settings the job used to export the data, the URLs where you can download the exported files again if needed, and the status of any currently running exports. To retrieve the history records for all export jobs, choose the option below:

Using the Interactive Documentation



1. Press the **GET /api/export/history** route to expand it.
2. Press **Try it out**.
3. (Optional) To return the history records for a specific export job rather than all records for all jobs, enter the ID of the export job into the **id** parameter.

   > **Note**: Use the Retrieve a List of All of Your Export Jobs route to find the `id` value for the job.

4. Press **Execute**.

Using a Third-Party Application

1. Create a `GET` request with the `/api/export/history` route and the CallMiner Bulk Export API URL.
2. (Optional) To return the history records for a specific export job rather than all records for all jobs, append the ID of the export job onto the end of the route URL. For example: *GET /api/export/history/5678* (where *5678* is the ID of the job).

   > **Note**: Use the Retrieve a List of All of Your Export Jobs route to find the `id` value for the job.

3. Set the following request headers:
   - **Authorization**: Set the value to the word *Bearer* followed by a space and then the authorization token. For example, *Bearer abc1234*.
   - **Content-Type**: Set the value to *application/json; charset=utf-8*.

   For example:

   ```
   GET /api/export/history
   Host: Bulk_Export_API_URL
   Authorization: Bearer abc1234
   Content-Type: application/json;
   charset=utf-8
   ```

4. Send the request.

The Response

> **Important**: If an error occurs, an HTTP status code in the `400` or `500` range is returned. Refer to the message provided with the error and HTTP Status Codes to troubleshoot what went wrong, correct the request, and try again.

If the request is successful, you will receive a `200` HTTP status code and a response body that contains the history of when each export job delivered data and the settings that each job ran on. The response fields you may be interested in are:

- **ExportJobId** (string): The unique identifier of the export job.
- **Id** (string): The unique identifier for this particular data export made by the larger export job.

  > **Example**: Let's say you have a recurring export job that delivers data every Monday at 12 A.M. The data export delivered on the first Monday will have its own ID, the data export delivered on the second Monday will have a separate ID, etc. The ID for each data export can be used for tracking purposes.

- **Status** (enum): The status of the export job. The status can be:
  - *InProgress*: The job is currently running and in the process of exporting the data.
  - *Failed*: The job has failed to run. Please follow the steps outlined in the Troubleshooting section to troubleshoot what went wrong and retrieve the exported data. If the issue persists, please contact support@callminer.com.
  - *Completed*: (Only applies to single-run jobs) The job has run successfully and finished exporting the requested data.
- **JobCompletionTime** (datetime): The date and time (in UTC) when the job finished exporting the data.
- **StorageTargetName** (string): The `Name` of the third-party storage target where your exported data was delivered, if applicable.
- **DownloadUrl** (string): The URL for the user interface where you can download the main CSV file (which contains each data type CSV file and the associated JSON-formatted schema file(s)) that the job originally exported and delivered. Paste this URL into a browser window to be redirected to a download page where your download will begin immediately.

> **Important**: If you chose to use a third-party storage target instead of email or webhook delivery, this value will be the path where your data was delivered within your third-party storage target.

- **DownloadEndpoint** (string): The API endpoint you can program against within your own application to download the main CSV file (which contains each data type CSV file and the associated JSON-formatted schema file(s)) that the job originally exported and delivered. Please note that we only recommend using this endpoint URL if you are programming against the Bulk Export API with your own application. The interactive documentation for this endpoint may not be able to handle downloading large file sizes.

  > **Important**: If you chose to use a third-party storage target instead of email or webhook delivery, this value will be the path where your data was delivered within your third-party storage target.

For a reminder of what the other fields in the response mean, check out the Bulk Export API interactive documentation.

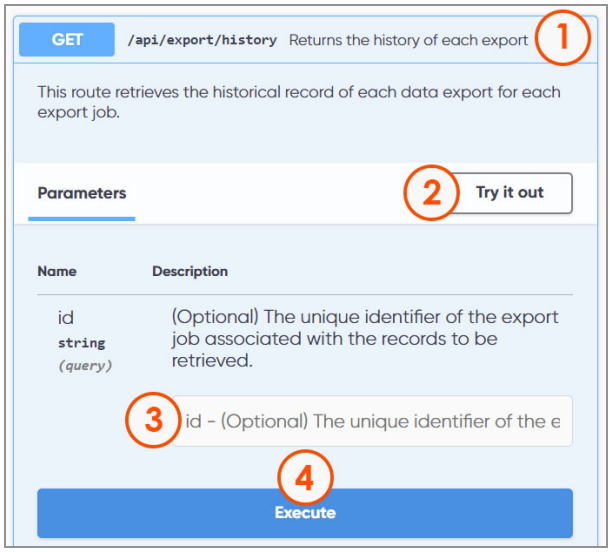
2. 8 | Download the Files from a Specific Export

Every time an export job delivers data, you'll receive an email or a webhook event where you can download the zipped CSV that contains your contact data CSV files (and the JSON-formatted schema files), but you can always use the API to download the data again if needed.

> **Note**: To quickly and easily download your files, we recommend using the **GET /api/export/history** route. With the **GET /api/export/history** route, we've provided you with two different options for downloading your files depending on your use-case:
>
> 1. **UI Download**: A quick and easy user interface that lets you download your files. Just find the `DownloadUrl` value in the response and paste it into your browser window to be redirected to a download page where your download will begin immediately.
> 2. **API Endpoint Download**: The API endpoint you can program against within your own application to download the specific files. Just find the `DownloadEndpoint` value in the response and use that to download the files into your application. Please note that we only recommend using this endpoint URL if you are programming against the Bulk Export API with your own application. The interactive documentation for this endpoint may not be able to handle downloading large file sizes.

To download the files, choose the option below you'd like to use:

Using the Interactive Documentation



1. Press the **GET /api/export/history** route to expand it.
2. Press **Try it out**.
3. Enter the ID of the export job associated with the files you want to download into the **id** parameter.

   > **Note**: Use the Retrieve a List of All of Your Export Jobs route to find the `id` value for the job.

4. Press **Execute**.

Using a Third-Party Application

1. Create a `GET` request with the `/api/export/history` route, the CallMiner Bulk Export API URL, and the ID of the export job associated with the files you want to download. For example: *GET /api/export/history/5678* (where *5678* is the ID of the job).

   > **Note**: Use the Retrieve a List of All of Your Export Jobs route to find the `id` value for the job.

2. Set the following request headers:
   - **Authorization**: Set the value to the word *Bearer* followed by a space and then the authorization token. For example, *Bearer abc1234*.
   - **Content-Type**: Set the value to *application/json; charset=utf-8*.

   For example:

   ```
   GET /api/export/history/5678
   Host: Bulk_Export_API_URL
   Authorization: Bearer abc1234
   Content-Type: application/json;
   charset=utf-8
   ```

3. Send the request.

The Response

> **Important**: If an error occurs, an HTTP status code in the `400` or `500` range is returned. Refer to the message provided with the error and HTTP Status Codes to troubleshoot what went wrong, correct the request, and try again.

If the request is successful, you will receive a `200` HTTP status code and a response body that contains a history record for each time the export job delivered data. Navigate to the particular history record associated with the files you want to download using the following response fields:

- **Id** (string): The unique identifier for this particular data export made by the larger export job.

  > **Example**: Let's say you have a recurring export job that delivers data every Monday at 12 A.M. The data export delivered on the first Monday will have its own ID, the data export delivered on the second Monday will have a separate ID, etc. The ID for each data export can be used for tracking purposes.

- **JobCompletionTime** (datetime): The date and time (in UTC) when the job finished exporting the data.

Once you've found the specific history record associated with the files you want to download, you have two different download field options:

- **DownloadUrl** (string): The URL for the user interface where you can download the main CSV file (which contains each data type CSV file and the associated JSON-formatted schema file(s)) that the job originally exported and delivered. Paste this URL into a browser window to be redirected to a download page where your download will begin immediately.

  > **Important**: If you chose to use a third-party storage target instead of email or webhook delivery, this value will be the path where your data was delivered within your third-party storage target.

- **DownloadEndpoint** (string): The API endpoint you can program against within your own application to download the main CSV file (which contains each data type CSV file and the associated JSON-formatted schema file(s)) that the job originally exported and delivered. Please note that we only recommend using this endpoint URL if you are programming against the Bulk Export API with your own application. The interactive documentation for this endpoint may not be able to handle downloading large file sizes.

> **Important**: If you chose to use a third-party storage target instead of email or webhook delivery, this value will be the path where your data was delivered within your third-party storage target.

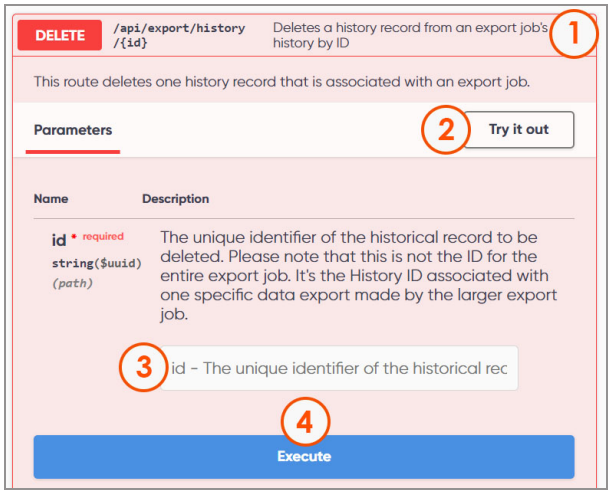
> **Note**: Only exports with a `Status` of *Completed* can be downloaded. If the `Status` is *InProgress*, the job is currently running and in the process of exporting the data. Please wait until the job has finished and try again. If your export has *Failed*, please follow the steps outlined in the Troubleshooting section to troubleshoot what went wrong and retrieve the exported data. If the issue persists, please contact support@callminer.com.

For a reminder of what the other fields in the response mean, check out the Bulk Export API interactive documentation.

2. 9 | Delete an Export History Record

A history record is generated every time a job exports data. You may want to delete history records once they are no longer needed to declutter the history you see for a job. To delete a history record, choose the option below you'd like to use and follow the steps outlined there:

Using the Interactive Documentation



1. Press the **DELETE /api/export/history/{id}** route to expand it.
2. Press **Try it out**.
3. Enter the ID of the history record you want to delete into the **id** parameter.

   > **Note**: Use the Retrieve the History of Data Exports for All Export Jobs route to find the `id` value for the record.

4. Press **Execute**.

Using a Third-Party Application

1. Create a `DELETE` request with the `/api/export/history/{id}` route, the CallMiner Bulk Export API URL, and the ID of the history record you want to delete. For example: *DELETE /api/export/history/5678* (where *5678* is the ID of the record).

   > **Note**: Use the Retrieve the History of Data Exports for All Export Jobs route to find the `id` value for the record.

2. Set the following request headers:
   - **Authorization**: Set the value to the word *Bearer* followed by a space and then the authorization token. For example, *Bearer abc1234*.
   - **Content-Type**: Set the value to *application/json; charset=utf-8*.

   For example:

   ```
   DELETE /api/export/history/5678
   Host: Bulk_Export_API_URL
   Authorization: Bearer abc1234
   Content-Type: application/json;
   charset=utf-8
   ```

3. Send the request.

The Response

> **Important**: If an error occurs, an HTTP status code in the 400 or 500 range is returned. Refer to the message provided with the error and HTTP Status Codes to troubleshoot what went wrong, correct the request, and try again.

If the request is successful, you will receive a 200 HTTP status code. You have successfully deleted the history record.

Appendix A

CallMiner Bulk Export API URLs

Interactive Documentation URL

Find your location below and use the appropriate link to access the CallMiner Bulk Export API:

Location	Bulk Export API URL
United States (Primary)	https://api.callminer.net/bulkexport/swagger
United States (FISMA/HITRUST)	https://apif.callminer.net/bulkexport/swagger
United Kingdom	https://apiuk.callminer.net/bulkexport/swagger
Australia	https://apiaus.callminer.net/bulkexport/swagger
Canada	https://apica.callminer.net/bulkexport/swagger
Europe (EU)	https://apiew.callminer.net/bulkexport/swagger

URL for Third-Party Application Use

Find your location below and use the appropriate link to make requests against the CallMiner Bulk Export API:

Location	Bulk Export API URL
United States (Primary)	https://api.callminer.net/bulkexport
United States (FISMA/HITRUST)	https://apif.callminer.net/bulkexport
United Kingdom	https://apiuk.callminer.net/bulkexport
Australia	https://apiaus.callminer.net/bulkexport
Canada	https://apica.callminer.net/bulkexport
Europe (EU)	https://apiew.callminer.net/bulkexport

Appendix B

CallMiner's Identity Provider Service URLs

The CallMiner Bulk Export API uses CallMiner's Identity Provider Service, an authentication application that employs industry-standard methods for user authentication and application authorization. With this authentication process, you use a Client ID and Secret to request an access token from the Identity Provider. You can then use this token in the `Authorization` header in whatever third-party application you want to use to authenticate each request you make against the CallMiner Bulk Export API.

Find your location below and use the appropriate URL to request a token in the "Authenticating with a Third-Party Application" section of Log In and Authenticate with the API:

Location	CallMiner's Identity Provider Service URL
United States (Primary)	https://idp.callminer.net
United States (FISMA/HITRUST)	https://idpf.callminer.net
United Kingdom	https://idpuk.callminer.net
Australia	https://idpaus.callminer.net
Canada	https://idpca.callminer.net
Europe (EU)	https://idpew.callminer.net

Appendix C

Managing Third-Party Storage Targets

The CallMiner Bulk Export API allows you to have your contact data delivered to a third-party storage target (such as Amazon AWS S3) instead of to webhook or email.

> **Important**: To use this feature, you must first fully configure the storage target location in your chosen third-party system where you want your Bulk Export data delivered.

Once you've created it in your third-party system, you can use the following sections to add the new storage location to the Bulk Export API and manage it:

Create or Update a Third-Party Storage Target 58
Retrieve Existing Storage Target Information 62
Delete a Third-Party Storage Target from Bulk Export 66

Create or Update a Third-Party Storage Target

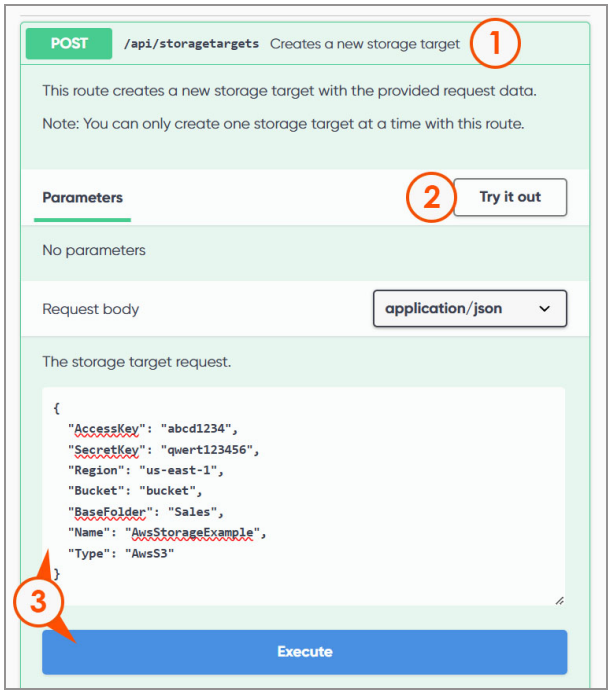
> **Important**: To use the routes in this section to create or update a third-party storage location (such as Amazon AWS S3) in the Bulk Export API, you must first fully configure your storage target location in your chosen third-party system.

> **Important**: Amazon AWS S3 is the only third-party storage target currently supported.

To create or update a third-party storage target, choose the option below you'd like to use and follow the steps outlined there:

> **Important**: We're going to use the **POST /api/storagetargets** route in our example to create a new storage target, but know that the **PUT** update route works the same way (on a storage target that already exists in the Bulk Export API) with the addition of the `Name` query parameter.

Using the Interactive Documentation



1. Press the route you want to use to expand it.
2. Press **Try it out**.
   a. (Only if using the **PUT /api/storagetargets/{storageTargetName}** route) Enter the `Name` of the storage target you want to retrieve into the `storageTargetName` query parameter.
3. Choose the third-party storage type you want to create or update below, enter the associated information into the request body, edit the payload values, and press **Execute**:

Using a Third-Party Application

1. Create a `POST` or `PUT` request with the route you want to use and the CallMiner Bulk Export API URL.
2. (Only if using the **PUT /api/storagetargets/{storageTargetName}** route) replace the `{storageTargetName}` parameter in the URL with the `Name` of the storage target you want to update. For example: *PUT /api/storagetargets/ExampleTarget* (where *ExampleTarget* is the `Name` value).
3. Set the following request headers:
   - **Authorization**: Set the value to the word *Bearer* followed by a space and then the authorization token. For example, *Bearer abc1234*.
   - **Content-Type**: Set the value to *application/json; charset=utf-8*.

   For example:

```
POST /api/storagetargets
Host: Bulk_Export_API_URL
Authorization: Bearer abc1234
Content-Type: application/json;
charset=utf-8
```

4. Choose the third-party storage type you want to create or update below, enter the associated information into the request body, edit the payload values, and send the request:

Amazon AWS S3

```
[
    {
        "AccessKey": "12345",
        "SecretKey": "67890",
        "Region": "us-east-1",
        "Bucket": "Example Bucket",
        "BaseFolder": "FolderName",
        "Name": "AWS Storage Example",
        "Type": "AwsS3"
    }
]
```

- **AccessKey** (string): ***Required***. The access key for the target location in S3.

  > **Important**: This information will be encrypted once you send the request.

- **SecretKey** (string): ***Required***. The secret key for the target location in S3.

  > **Important**: This information will be encrypted once you send the request.

- **Region** (string): ***Required***. The region of the target location in S3.
- **Bucket** (string): ***Required***. The name of the bucket of the target location in S3.
- **BaseFolder** (string): Optional. The name of the folder in the bucket in S3.
- **Name** (string): ***Required***. The name of the storage target.

  > **Important**: This value is the unique identifier for the storage target. You will enter this `Name` value into the `StorageTargetName` value in the **POST api/export/job** route to create a job that will deliver your data to this third-party storage location.

> **Note**: If you update the name of an existing storage target, all related export jobs will be updated automatically to use the new name.

- **Type** (enum): ***Required***. Enter *AwsS3*.

The Response

> **Important**: If an error occurs, an HTTP status code in the `400` or `500` range is returned. Refer to the message provided with the error and HTTP Status Codes to troubleshoot what went wrong, correct the request, and try again.

After you send the request, if the request is successful, you will receive a `201` (if using the **POST** route) or a `200` (if using the **PUT** route) HTTP status code and a response body that contains all of the information you sent in with the request (except for the sensitive values, which will appear as empty strings in the response).

Retrieve Existing Storage Target Information

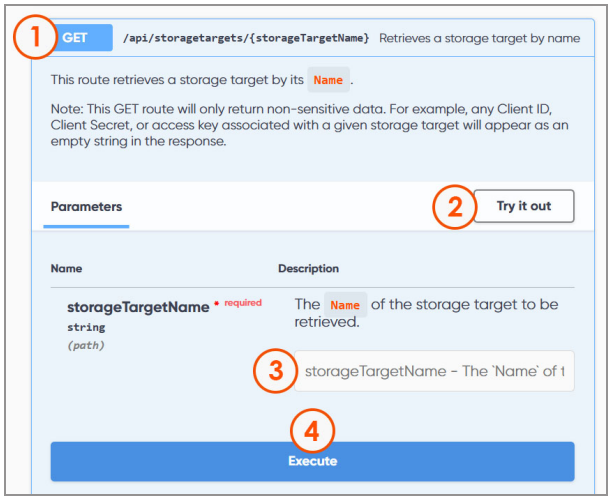
You have two different options when retrieving storage target information:

1. To retrieve a single storage target by name, use the **GET /api/storagetargets/{storageTargetName}** route.
2. To retrieve a list of all storage targets that you've created in the Bulk Export API, use the **GET /api/storagetargets** route.

To retrieve the information, complete the following:

> **Important**: We're going to use the **GET /api/storagetargets/{storageTargetName}** route in our example, but know that the **GET /api/storagetargets** route works the same way, minus the `Name` query parameter.

Using the Interactive Documentation



1. Press the **GET** route you want to use to expand it.
2. Press **Try it out**.
3. (Only if using the **GET /api/storagetargets/{storageTargetName}** route) Enter the **`Name`** of the storage target you want to retrieve into the **`storageTargetName`** parameter.
4. Press **Execute**.

Using a Third-Party Application

1. Create a **`GET`** request with the route you want to use and the CallMiner Bulk Export API URL.
2. (Only if using the **GET /api/storagetargets/{storageTargetName}** route) replace the **`{storageTargetName}`** parameter in the URL with the **`Name`** of the storage target you want to retrieve. For example: *GET /api/storagetargets/ExampleTarget* (where *ExampleTarget* is the **`Name`** value).
3. Set the following request headers:
   - **Authorization**: Set the value to the word *Bearer* followed by a space and then the authorization token. For example, *Bearer abc1234*.
   - **Content-Type**: Set the value to *application/json; charset=utf-8*.

   For example:

   ```
   GET
   /api/storagetargets/ExampleTar
   get
   Host: Bulk_Export_API_URL
   Authorization: Bearer abc1234
   Content-Type: application/json;
   charset=utf-8
   ```
4. Send the request.

The Response

> **Important**: If an error occurs, an HTTP status code in the **400** or **500** range is returned. Refer to the message provided with the error and HTTP Status Codes to troubleshoot what went wrong, correct the request, and try again.

If the request is successful, you will receive a **200** HTTP status code and a response body that contains information about the specific storage target you requested or all storage targets that you created in the system, depending on which route you chose to use.

With each storage target in the response, you'll receive the same information depending on which third-party storage system the target is associated with. To understand the values returned in the response, see the following:

Amazon AWS S3

```
[
    {
        "AccessKey": "",
        "SecretKey": "",
        "Region": "us-east-1",
        "Bucket": "Example Bucket",
        "BaseFolder": "FolderName",
        "Name": "AWS Storage Example",
        "Type": "AwsS3"
    }
]
```

- **AccessKey** (string): Because this is sensitive information, this route will always return this value as an empty string.
- **SecretKey** (string): Because this is sensitive information, this route will always return this value as an empty string.
- **Region** (string): The region of the target location in S3.
- **Bucket** (string): The name of the bucket of the target location in S3.
- **BaseFolder** (string): The name of the folder in the bucket in S3.

- **Name** (string): The name of the storage target.

> **Important**: This value is the unique identifier for the storage target. You can enter this `Name` value into:
>
> - The **PUT api/storagetargets/{storageTargetName}** route to update the storage target.
> - The **DELETE api/storagetargets/{storageTargetName}** route to delete the storage target.
> - The `StorageTargetName` value in the **POST api/export/job** route to create a job that will deliver your data to this third-party storage location.

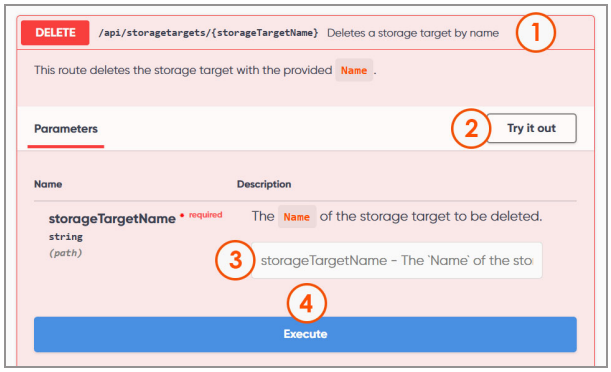
- **Type** (enum): Will always return a value of *AwsS3* for all Amazon AWS S3 storage targets.

Delete a Third-Party Storage Target from Bulk Export

To delete one of your storage targets, choose the option below you'd like to use and follow the steps outlined there:

> **Important**: You cannot delete a storage target if one or more of your export jobs are currently delivering to that storage target. You must either delete the export jobs or update them so they use another storage target before using this route for deletion.
>
> Use the Retrieve a List of All of Your Export Jobs route to verify that no current export job is using the `StorageTargetName` that you want to delete before proceeding.

Using the Interactive Documentation



1. Press the **DELETE /api/storagetargets/{storageTargetName}** route to expand it.
2. Press **Try it out**.
3. Enter the **Name** of the storage target you want to delete into the **storageTargetName** parameter.

   > **Note**: Use the Retrieve Existing Storage Target Information section to find the **Name** value for the storage target you want to delete.

4. Press **Execute**.

Using a Third-Party Application

1. Create a **DELETE** request with the **/api/storagetargets/{storageTargetName}** route, the CallMiner Bulk Export API URL, and the **Name** of the storage target you want to delete. For example: *DELETE /api/storagetargets/Target1 (where Target1 is the **Name** value)*.

   > **Note**: Use Retrieve Existing Storage Target Information to find the **Name** value for the storage target you want to delete.

2. Set the following request headers:
   - **Authorization**: Set the value to the word *Bearer* followed by a space and then the authorization token. For example, *Bearer abc1234*.
   - **Content-Type**: Set the value to *application/json; charset=utf-8*.

   For example:

   ```
   DELETE
   api/storagetargets/Target1
   Host: Bulk_Export_API_URL
   Authorization: Bearer abc1234
   Content-Type: application/json;
   charset=utf-8
   ```

3. Send the request.

The Response

> **Important**: If an error occurs, an HTTP status code in the 400 or 500 range is returned. Refer to the message provided with the error and HTTP Status Codes to troubleshoot what went wrong, correct the request, and try again.

If the request is successful, you will receive a 200 HTTP status code. You have successfully deleted the storage target. It will no longer run or exist in the system.

Appendix D

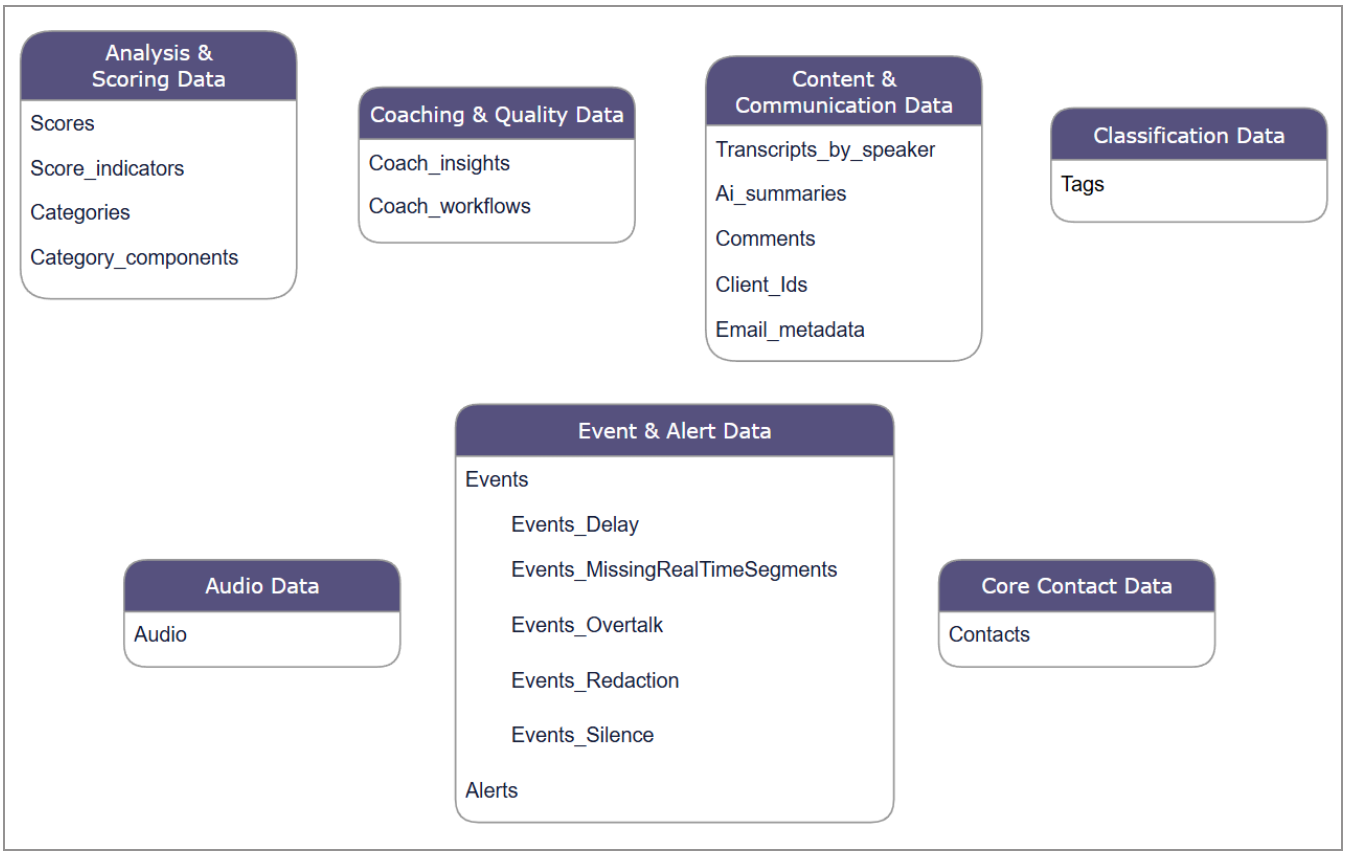
Export Data Types

When exporting your contact data, you can choose to export a variety of data types to CSV.

Here is a diagram representing the different data types and how they relate to one another:

> **Important**: Each data type you export will be organized by Contact ID.

> **Note**: When you choose the `Events` data type, you will receive a separate layout file and CSV for each of the event types listed beneath it in the chart.



Below, you'll find more information about each data type and what will be included with it in your exports:

Note: If you choose to include multiple data types in your export, each `DataType` below will be exported into its own zipped CSV file within the main zip file that you download. Each CSV file will be organized by Contact ID. For example, if you choose to export alerts and categories, the Alerts CSV file will list each Contact ID with the alerts that fired on each contact, and the Categories CSV file will list each Contact ID with the categories that hit for each contact.

Important: The `Audio` data type is the only exception to this rule. If you export audio, you will receive a CSV file within the main zip file that provides a `Status` for each exported audio file. Each audio file itself will be delivered to the path you designated in your third-party storage target.

Note: The `ResultMessage` and `RecordCount` in the layout file provided with each exported `DataType` will indicate how many individual records were exported of that type. For example, when exporting *14407* AI Summaries, you could receive something like the following:

```
"RecordCount": 14407,
"ResultMessage": "Successfully returned 14407 record(s)",
```

If no records matching the requested data type were found, you'll receive the following:

```
"ResultMessage": "Sorry, we could not find any data that matched the filters on
your export job. Please create a new export job with different filters and try
again. If the issue persists, please contact CallMiner support."
```

Important: We do not recommend exporting your data to Excel as there is a limit on the number of rows that can be included in each file and on the number of characters allowed in each cell. If the data in your export exceeds either limit, you may experience issues in Excel with the formatting and content of your file(s).

Data Type	API DataType Value	Definition	What Gets Exported
AI Summaries	`Ai_summaries`	AI summaries are contact summaries that	For each AI summary: • **ContactID** (integer): The

Data Type	API DataType Value	Definition	What Gets Exported
		have been generated by artificial intelligence. They provide a brief, high-level overview of what occurred in a contact. Please note that Contact Summary is a paid add-on. For information on purchasing Contact Summary, please contact your CallMiner sales representative.	associated Contact ID. • **Summary** (string): The AI summary text. • **ActionItems** (string): The action items associated with the contact and summary. • **Reason** (string): Will return a value of `AI Summary not found` as an explanation if the **`Summary`** column is *NULL*. If an actual **`Summary`** value is returned, this column will be empty.
Alerts	**`Alerts`**	Alerts are real-time messages or notifications that fire when specified words or events occur within a contact. An alert is comprised of one or more components that dictate what language, metadata filters, or even silence should cause the alert to fire. Please note that alerts require the RealTime feature and RealTime is a paid add-on. For information on purchasing RealTime, please contact your CallMiner sales representative.	For each alert that fired: • **ContactID** (integer): The associated Contact ID. • **AlertID** (integer): The unique identifier of the alert. • **AlertName** (string): The name of the alert. • **TimeStamp** (datetime): The date and time the alert fired during the contact (in UTC).

Data Type	API DataType Value	Definition	What Gets Exported
Audio	`Audio`	Audio is the audio file (in MP3 or OGG format) associated with an audio or video contact. **Important**: You can only use this data type if you are exporting your files to a third-party storage target. See Managing Third-Party Storage Targets for more information on how to set one up.	The audio file for each contact. The layout file associated with the audio export will also include: • **ContactID** (integer): The associated Contact ID. • **Path** (string): The path in your third-party storage target where your audio files were delivered. • **Status** (string): The upload status of each audio file.
Categories	`Categories`	Categories are tools that combine multiple facets of a search -- using metadata, filters, key words and phrases, even acoustics -- all rolled together to be used to perform hard-hitting analyses of your contacts.	For each category that hit: • **ContactID** (integer): The associated Contact ID. • **CategoryID** (integer): The unique identifier of the category. (**Note**: The equivalent field in the CallMiner Data API was called `BucketId`.) • **CategoryFullName** (string): The full name of the category, including the section the category belongs to. (**Note**: The equivalent field in the CallMiner Data API was called `BucketFullname`.) • **CategoryName** (string): The

Data Type	API DataType Value	Definition	What Gets Exported
			name of the category. (**Note**: The equivalent field in the CallMiner Data API was called `BucketName`.) • **SectionID** (integer): The unique identifier of the section the category belongs to. • **SectionName** (string): The name of the section the category belongs to.
Category Components	`Category_components`	Category components are individual levels of a category. They can be comprised of a piece of metadata or a phrase search, etc. For example, the **Acknowledgment** category might have a component that looks for the phrase *I understand*.	For each category component that hit: • **ContactID** (integer): The associated Contact ID. • **ComponentID**(integer): The unique identifier of the component. • **ComponentName** (string): The name of the component. • **CategoryID** (integer): The unique identifier of the category the component belongs to. (**Note**: The equivalent field in the CallMiner Data API was called `BucketId`.) • **CategoryFullName** (string): The full name of the category the component belongs to. This full name includes the name of the category and the section the category belongs to.

Data Type	API DataType Value	Definition	What Gets Exported
			(**Note**: The equivalent field in the CallMiner Data API was called `BucketFullname`.) • **CategoryDescription** (string): The description of the category. (**Note**: The equivalent field in the CallMiner Data API was called `BucketDescription`.) • **CategoryName** (string): The name of the category that the component belongs to. (**Note**: The equivalent field in the CallMiner Data API was called `BucketName`.) • **SectionName** (string): The name of the section the category and its components belong to. • **StartTime** (decimal): The start time of the component hit (in seconds). • **EndTime** (decimal): The end time of the component hit (in seconds). • **Weight** (decimal): The weight of the component. • **SimilarityScore** (single): The AI similarity score of the component hit (only applies to components that belong to AI categories).

Data Type	API DataType Value	Definition	What Gets Exported
Client IDs	`Client_Ids`	A Client ID is a unique identifier that you provide to identify each individual contact in your system. All export files are organized by Contact ID, which is the unique identifier the CallMiner system assigns to each contact. Use this export data type to map between the CallMiner-assigned Contact ID and the Client ID you designated for each contact in your system.	For each contact: • **ContactID** (integer): The associated Contact ID. • **ClientID** (string): The Client ID for the contact. • **ClientCaptureDate** (datetime): The date and time (in local time) when the contact originally occurred.
Coach Insights	`Coach_insights`	Coach insights allow users to share a contact with another user for the purposes of highlighting certain behavior or occurrences on the contact, either commendable or coachable. For example, a coach might send an agent an insight on a contact where they missed giving the required greeting to coach them and remind them to give the greeting the	For each insight: • **ContactID** (integer): The associated Contact ID. • **InsightID** (string): The unique identifier of the insight. • **AssignedByUser** (string): The email address of the user who assigned the insight. • **AssignedBy** (string): The name of the user who assigned the insight. • **CreationDate** (datetime): The date and time (in UTC) when the insight was created. • **AssignedToEmail** (string): The email address of the user the

Data Type	API DataType Value	Definition	What Gets Exported
		next time they handle a contact. Please note that Coach is a paid add-on. For information on purchasing Coach, please contact your CallMiner sales representative.	insight was assigned to. • **AssignedTo** (string): The name of the user the insight was assigned to. • **OriginalDueDate** (datetime): The original due date for the insight (in UTC) (only applies if the due date has been changed). • **DueDate** (datetime): The current due date for the insight. • **CompletedDate** (datetime): The date and time (in UTC) when the insight was completed. • **Indicators** (string): The score indicators associated with the insight. • **Status** (string): The current status of the insight. Can be *Active*, *Overdue*, *Complete*, or *Canceled*. • **CurrentOwner** (string): The username of the current owner of the insight. • **LastUpdated** (datetime): The date and time (in UTC) when the insight was last updated. • **Acknowledged** (boolean): Whether the insight was acknowledged.

Data Type	API DataType Value	Definition	What Gets Exported
			• **FirstListenedTime** (datetime): The date and time (in UTC) when the audio associated with the insight was first listened to. • **ContactType** (string): The type of contact. Can be *call*, *chat*, *email*, etc. • **ContactDate** (datetime): The date and time (in local time) when the contact originally occurred. • **AgentEmail** (string): The email address of the agent on the insight. • **DisplayDescriptionsInCoach** (boolean): Whether the score associated with the insight is displayed as its bin descriptions or the score value in Coach. If *true*, the score is displayed as the bin descriptions. If *false*, the score is displayed as the score value. • **SnippetStart** (integer): The number of seconds into the contact where the audio snippet associated with the insight starts. (Only applies to insights that have been sent with a specific snippet of a contact instead of the whole contact).

Data Type	API DataType Value	Definition	What Gets Exported
			• **SnippetEnd** (integer): The number of seconds into the contact where the audio snippet associated with the insight ends, if applicable. • **ContactLength** (integer): The number of seconds that comprise the whole contact. • **ScoreID** (integer): The unique identifier of the score associated with the insight. • **ScoreName** (string): The name of the score associated with the insight. • **OriginalScoreValue** (double): The original value for the score (only applies if the value has been changed). • **ScoreValue** (double): The current value for the score. • **CommentDate** (datetime): The date and time (in UTC) of a comment on the insight. • **CommentCreatorEmail** (string): The email address of the user who created the comment. • **CommentCreatorName** (string): The name of the user who created the comment. • **Comments** (string): The text of the comments on the insight. • **Type** (string): The type of

Data Type	API DataType Value	Definition	What Gets Exported
			insight. • **CompletedByEmail** (string): The email address of the user who completed the insight. • **CompletedBy** (string): The name of the user who completed the insight. • **Forwarded** (string): Whether the insight was forwarded to another user. If *Yes*, the insight was forwarded. If *No*, it's an original insight. • **OriginalInsightID** (string): The Insight ID of the original insight (only applies if this insight was forwarded and given a new ID).
Coach Workflows	`Coach_workflow`	Coach workflows are events where the owner or status changed for a contact in Coach. For example, a user might change the status of a contact from *Open* to *In Progress* and change the owner of the contact from one Coach user to another. Please note that Coach is a paid add-on. For information on purchasing Coach, please contact your	For each workflow: • **ContactID** (integer): The associated Contact ID. • **ID** (string): The unique identifier of the workflow. • **NewOwner** (string): The new owner of the workflow, if the owner has changed. • **NewStatus** (string): The new status of the workflow, if it has been updated. • **OldOwner** (string): The previous owner. • **OldStatus** (string): The previous

Data Type	API DataType Value	Definition	What Gets Exported
		CallMiner sales representative.	status. • **Timestamp** (datetime): The date and time (in UTC) when the workflow status and/or owner changed. • **UserName** (string): The username of the user making the workflow change. • **ChangeType** (string): The type of change.
Comments	`Comments`	The comments and replies that have been posted on a contact in Analyze.	For each comment: • **ContactID** (integer): The associated Contact ID. • **CommentID** (integer): The unique identifier of the comment. • **Comment** (string): The text that makes up the comment. • **CreationDate** (datetime): The date and time (in UTC) when the comment was created. • **Author** (string): The author of the comment. • **LastUser** (string): The user who last modified the comment. • **LastModified** (datetime): The date and time (in UTC) when the comment was last modified.
Contacts	`Contacts`	A contact is an individual customer	For each contact:

Data Type	API DataType Value	Definition	What Gets Exported
		interaction that has been analyzed. Contacts can be phone calls, videos, emails, tweets, chats, etc.	• **ID** (integer): The associated Contact ID. • **LastModified** (datetime): The date and time (in UTC) when the contact was last modified. Use this value to track updates or maintain versioning of contacts in your system. • The identifying information and metadata fields associated with the contact, such as *Client Capture Date*, *Word Count*, *Agent*, etc. > **Note**: The information returned will vary based on system configuration. See Retrieve Metadata Information to find out what each field in your `Contacts` export means.
Email Contact Metadata	`Email_metadata`	Email contacts (one or more emails in a larger email thread) are a specific type of text-based contact, and, as such, they have different metadata items associated with them. This data type exports the metadata specific to email contacts.	For every email in each email contact thread: • **ContactID** (integer): The associated Contact ID. • **StartTime** (decimal): The start time (in seconds) that indicates when an individual email appears in a larger email contact thread. • **Sent** (datetime): The date and time (in UTC) when the email was sent.

Data Type	API DataType Value	Definition	What Gets Exported
			• **Subject** (string): The subject line of the email. • **From** (string): The email address of the user who sent the email. • **To** (string): The comma-delimited list of direct recipients the email was sent to. • **CC** (string): The CC recipients of the email (as a comma-delimited list). • **BCC** (string): The BCC recipients of the email (as a comma-delimited list). • **Attachments** (string): The file names of attachments on the email (as a comma-delimited list).
Events	`Events`	Events are things that occurred within a contact. They include overtalk events, redaction events, category hits, tag hits, alert hits, silence events, missing real-time audio events, and (only for text-based contacts) delay events.	For each event: • **ContactID** (integer): The associated Contact ID. • **StartTime** (decimal): The start time of the event (in seconds). • **EndTime** (decimal): The end time of the event (in seconds). • **Duration** (decimal): The duration of the event (in seconds). Furthermore, delay events will also include:

Data Type	API DataType Value	Definition	What Gets Exported
		Note: Each of the following event types will have their own layout file and CSV in the export: • Delay • Missing Real-Time Segments • Overtalk • Redaction • Silence	• **Speaker** (byte): The speaker associated with the delay event. • **PreviousSpeaker** (byte): The previous speaker associated with the delay event. Redaction events may also include: • **SpeakerID** (byte): The unique identifier of the speaker associated with the event. • **SpeakerName** (string): The name of the speaker associated with the event. **Note**: The following apply to AI Redaction events only. • **FriendlyName** (string): The friendly name of the redaction entity associated with the event. • **Text** (string): The text that the redacted words were replaced with in the contact's transcript for the event. • **EntityType** (string): The type of redaction entity associated with the event. • **Description** (string): The description of the redaction entity.

Data Type	API DataType Value	Definition	What Gets Exported
			• **RegulatoryCompliance**(array): The different regulatory compliance types that this redaction entity applies to.
Scores	`Scores`	Scores are tools that measure specific targets. For example, a contact might be scored on **Agent Quality**, **Customer Satisfaction**, or even measures such as **Call Duration**.	For each score: • **ContactID** (integer): The associated Contact ID. • **ScoreID** (integer): The unique identifier of the score. • **ScoreName** (string): The name of the score. • **Score** (double): The value the contact received for the score.
Score Indicators	`Score_indicators`	Score indicators are the individual levels of a score. For example, the **Agent Quality** score might have an indicator that scores an agent on *Politeness*.	For each score indicator the contact scored on: • **ContactID** (integer): The associated Contact ID. • **ScoreID** (integer): The unique identifier of the score the indicator belongs to. • **ScoreComponentID** (integer): The unique identifier of the indicator. • **ComponentFullName** (string): The full name of the indicator, which includes the name of the score the indicator belongs to. • **DisplayDescription** (boolean): Whether or not the

Data Type	API DataType Value	Definition	What Gets Exported
			description for the indicator is displayed in Coach or its value. If *true*, the indicator is displayed with its description. If *false*, the indicator is displayed with its value. • **CurrentlyActive** (boolean): Whether the indicator is currently active. • **DisplayFormat** (byte): The display format of the indicator. • **DisplayOrder** (integer): The order of this score indicator within the score. • **Value** (double): The value the contact received for the indicator.
Tags	`Tags`	Tags are tools that may be applied to a contact (either the entire contact or a portion within) to help identify and label points of interest to be easily searched for later. For example, a supervisor might add a tag to a contact called **Review Later** that marks the contacts they need to go back to and review at a later date.	For each tag applied to a contact: • **ContactID** (integer): The associated Contact ID. • **TagID** (integer): The unique identifier of the tag. (**Note**: The equivalent field in the CallMiner Data API was called `BucketId`.) • **TagName** (string): The name of the tag. (**Note**: The equivalent field in the CallMiner Data API was called `BucketName`.) • **TagFullName** (string): The full name of the tag, which includes the name of the

Data Type	API DataType Value	Definition	What Gets Exported
			section the tag belongs to. (**Note**: The equivalent field in the CallMiner Data API was called `BucketFullname`.)
Speaker-Organized Transcripts	`Transcripts_by_speaker`	Transcripts are comprised of the words that were originally spoken or written by each speaker in the contact. Each transcript is organized by speaker turn (when one person spoke vs. when the other person responded). If the contact was originally an audio or video-based contact, the transcript is made up of the words that have been transcribed from the original audio from each speaker turn.	For each contact transcript: • **ContactID** (integer): The associated Contact ID. • **StartTime** (decimal): The start time (in seconds) of each speaker turn in the contact. • **SpeakerID** (byte): The unique identifier of the speaker associated with each speaker turn. • **SpeakerName** (string): The name of the speaker associated with each speaker turn. • **Text** (string): The text (or spoken words that have been transcribed into text) that makes up the speaker turn.
Transcripts ***(Deprecated)***	`Transcripts`	**Important**: This data type has been deprecated. Use the `Transcripts_by_speaker` data type instead.	**Important**: This data type has been deprecated. Use the `Transcripts_by_speaker` data type instead.

Appendix E

JSON-Formatted Schema File Contents

When the Bulk Export API exports your contact data, you will receive a CSV file for each data type you requested in your export. Along with the CSV file(s), you will also receive a JSON-formatted schema file for each data type that contains further information about your export.

> **Note**: When you receive your exported contact data, you will receive a main zip file that contains the JSON-formatted schema file(s) for each requested data type, and within that main zip file, you will find a Gzipped CSV file for each data type (containing the actual contact data).

> **Note**: The `Audio` data type is the only exception to this rule. If you export audio, you will receive a CSV file within the main zip file that provides a `Status` for each exported audio file. Each audio file itself will be delivered to the path you designated in your third-party storage target.

Here is an example of what the JSON-formatted schema file for a particular data type might look like:

```
{
  "ExportId": "64dcdac8f31710e24ccbe2d2",
  "ExportRuntime": "00:02:48.5267632",
  "DataRange": {
    "LastNDays": null,
    "TimeFrame": "Custom",
    "StartDate": "2023-04-10T12:15:45.5432109Z",
    "EndDate": "2023-04-13T23:59:59Z"
  },
  "ExportDate": "2023-05-11T14:21:54.3306796Z",
  "RecordCount": 38810,
  "ResultMessage": "Successfully returned 38810 record(s)",
  "LayoutEntity": [
    {
      "ColumnName": "Example 1",
      "FriendlyName": "Example ID",
```

```
      "Type": "System.Int32",
      "Ordinal": 1
    }
  ]
}
```

Within the schema file, each field provides the following information:

- **ExportId** (string): The unique identifier of this particular export.
- **ExportRuntime** (string): The amount of time it took for the Bulk Export system to export this data.
- **DateRange** (object): The timeframe that the data was exported from when the export job ran.
- **ExportDate** (datetime string): The date and time when the export was initiated or requested (in UTC).
- **RecordCount** (integer): The total number of contacts that were exported.
- **ResultMessage** (string): The message the Bulk Export system generates when the export is completed. If the export was successful, this message will indicate how many records (contacts) were successfully returned. If the export was not successful, this message will indicate that there was an issue with the export. Refer to the provided message to troubleshoot what went wrong, correct the issue, and try again.
- **LayoutEntity** (array of objects): This array lists each data attribute or field that appears in the exported CSV file(s). Each data or attribute will have its own column in the associated CSV file(s). Each object in this array represents one column and contains the following information for each column:
  - **ColumnName** (string): The name of the column in the database.
  - **FriendlyName** (string): The friendly name for the column that appears in the CSV file(s).
  - **Type** (string): The type of data that the column contains. For example, a column might have *string* as its type, which indicates that each value within the column will be in string format.
  - **Ordinal** (integer): The number corresponding to the place where the column appears in the exported CSV. For example, a value of *1* indicates that this is the first column in the CSV file. (Note that the first column in each CSV will always have an `Ordinal` value of *1*.)

Appendix F

Retrieve Metadata Information

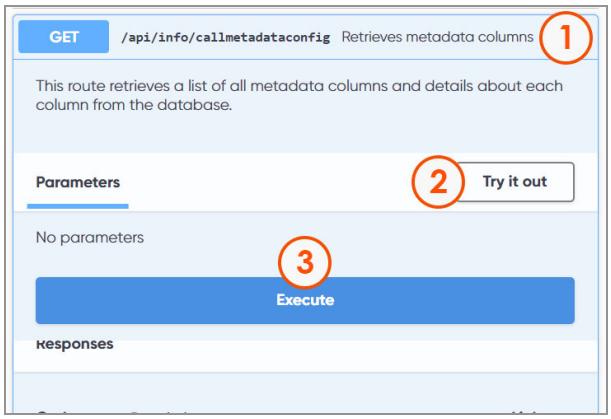
You can use a variety of metadata to filter the data that will be returned in a given export, and you can receive a variety of metadata on an export that has its `DataType` value set to *Contacts*.

To make things easier, we've included an informational route in the Bulk Export API to help you:

- Determine if a metadata item can be used as one of the `SearchFilters` on an export.
- Find the `ColumnName` of a metadata item to use it as one of the `SearchFilters` on an export.
- Understand what each piece of information means in an exported file that has *Contacts* as its `DataType` value.

To retrieve a list of all metadata items in your Eureka system:

Using the Interactive Documentation



1. Press the **GET /api/info/callmetadataconfig** route to expand it.
2. Press **Try it out**.
3. Press **Execute**.

Using a Third-Party Application

1. Create a **GET** request with the **/api/info/callmetadataconfig** route and the CallMiner Bulk Export API URL.
2. Set the following request headers:
   - **Authorization**: Set the value to the word *Bearer* followed by a space and then the authorization token. For example, *Bearer abc1234*.
   - **Content-Type**: Set the value to *application/json; charset=utf-8*.

   For example:

   ```
   GET
   /api/info/callmetadataconfig
   Host: Bulk_Export_API_URL
   Authorization: Bearer abc1234
   Content-Type: application/json;
   charset=utf-8
   ```
3. Send the request.

The Response

> **Important**: If an error occurs, an HTTP status code in the **400** or **500** range is returned. Refer to the message provided with the error and HTTP Status Codes to troubleshoot what went wrong, correct the request, and try again.

If the request is successful, you will receive a **200** HTTP status code and a response body that contains each metadata item in your Eureka system.

The following details the information included with each metadata item:

- **ColumnName** (string): The column name for the metadata item in the database. To filter your export by this metadata item, enter this as the `ColumnName` value within an object within the `SearchFilters` array in the Create a Job to Export Contact Data or Update a Recurring Export Job routes.

  > **Note**: The friendly name for a metadata item in Analyze may differ from the `ColumnName` for the metadata item. If you know the friendly name for the item from Analyze, look for the metadata item in this response that has that same value for the `FriendlyName` field. Then, note the `ColumnName` value for the item.

- **FriendlyName** (string): The user-facing name for the metadata item in Analyze or Coach. For example, if the column name for a metadata item is *WAVLength*, the friendly name in Analyze or Coach might be *Duration*.
- **Description** (string): The description for the metadata item.
- **DotNetType** (string): The C# type of the metadata item. For example, *System.DateTime* or *System.String*.
- **IsAdvancedFilter** (boolean): Indicates if the metadata item can be used as a filter in the `SearchFilters` array on either the Create a Job to Export Contact Data or Update a Recurring Export Job routes. If *true*, the metadata item can be used to filter the data returned in an export. If *false*, it cannot be used as a filter.
- **Ranges** (array of strings): The ranges that belong to the metadata item, if applicable. For example, the `Agitation` metadata item might have a *Low*, *Medium*, and *High* range.

  > **Note**: You can use the name of a range as the `Operand1` value in an object in the `SearchFilters` array to filter your export by every contact that either matched or didn't match that range. See the Create a Job to Export Contact Data `SearchFilters` section or SearchFilters Basics for more information.

Use the information you've found in this response to either understand the data you received from an export that had its `DataType` set to *Contacts*, or use the information to build out the `SearchFilters` on a given export and then return to either the Create a Job to Export Contact Data section or the Update a Recurring Export Job section to continue configuring your export job.

Appendix G

SearchFilters Basics

Filter Options

You have a variety of filter options to choose from when creating or updating an export job.

The **SearchFilters** array is used to filter the data returned in the export. This is what the base **SearchFilters** array looks like:

```
"SearchFilters": [
        {
          "CategoryId": null,
          "TagId": null,
          "SectionId": null,
          "ScoreId": null,
          "AlertId": null,
          "TableFilterTypes": null,
          "ColumnName": "Agent",
          "Operator": "=",
          "Operand1": "John Doe",
          "Operand2": null
        }
      ]
```

To set it, choose from the following filter options:

Option	How to Set It	Example

#1 Filter your export by Category ID(s).	Enter each **CategoryId** value you want to filter by into its own object within the **SearchFilters**	`"SearchFilters": [`
		`  {`
		`  "CategoryId": 12345,`
		`  "Operator": "="`
		`  },`
		`  {`

Option	How to Set It	Example

	array. (To obtain a list of **CategoryId** values in your Eureka system, use the Retrieve Content SearchFilters Values route.)
	
	Then, set **Operator** to = (equals).
		`"CategoryId": 67890,`
		`"Operator": "="`
		`}`
		`]`

#2 Filter your export by Tag ID(s).	Enter each **TagId** value you want to filter by into its own object within the **SearchFilters** array. (To obtain a list of **TagId** values in your Eureka system, use the Retrieve Content SearchFilters Values route.)
	
	Then, set **Operator** to = (equals).
		`"SearchFilters": [`
		`{`
		`"TagId": 573621,`
		`"Operator": "="`
		`},`
		`{`
		`"TagId": 48472,`
		`"Operator": "="`
		`}`
		`]

#3 Filter your export by Section ID(s).	Enter each **SectionId** value you want to filter by into its own object within the **SearchFilters** array. (To obtain a list of **SectionId** values in your Eureka system, use the Retrieve	`"SearchFilters": [`
		`{`
		`"SectionId": 12345,`
		`"Operator": "="`
		`},`
		`{`
		`"SectionId": 67890,`
		`"Operator": "="`
		`}`
		`]

Option	How to Set It	Example
	Content SearchFilters Values route.) Then, set **`Operator`** to = (equals).	

#4 Filter your export by Alert ID(s).	Enter each **`AlertId`** value you want to filter by into its own object within the **`SearchFilters`** array. (To obtain a list of **`AlertId`** values in your Eureka system, use the Retrieve Content SearchFilters Values route.)
	
	Then, set **`Operator`** to = (equals).
		`"SearchFilters": [`
		`  {`
		`  "AlertId": 12345,`
		`  "Operator": "="`
		`  },`
		`  {`
		`  "AlertId": 67890,`
		`  "Operator": "="`
		`  }`
		`  ]`

#5 Filter your export by contacts that fell into a specific score range based on the name of the range.	Enter the **`ScoreId`** value of the score the range belongs to in an object in the **`SearchFilters`** array. Then, set **`Operator`** to = (equals) and **`Operand1`** to the friendly name of the score range. (To obtain a list of **`ScoreId`** and score **`RangeFriendlyName`** values in your Eureka system, use	**Example**: You want to export every contact that fell into the *Poor* range of your **Politeness** score.
		
		`"SearchFilters": [`
		`  {`
		`  "ScoreId": 573621,`
		`  "Operator": "=",`
		`  "Operand1": "Poor"`
		`  }`
		`  ]`

Option	How to Set It	Example
	the Retrieve Content SearchFilters Values route.)	

#6 Filter your export by contacts that fell into a specific score range based on the actual range values.	Enter the **ScoreId** value of the score the range belongs to in an object in the **SearchFilters** array. Then, set **Operator** to *between*, **Operand1** to the lower value, and **Operand2** to the higher value you are looking for. (To obtain a list of **ScoreId** and score **RangeMin** and **RangeMax** values in your Eureka system, use the Retrieve Content SearchFilters Values route.)	**Example**: You want to export every contact that scored anywhere from *0* to *50* on your **Politeness** score.
		
		`"SearchFilters": [`
		`  {`
		`  "ScoreId": 573621,`
		`  "Operator": "between",`
		`  "Operand1": "0",`
		`  "Operand2": "50"`
		`  }`
		`  ]`

#7 Filter your export by one or more consecutive words within the comments on each contact.	Set the **TableFilterTypes** field to *CallComment* within the **SearchFilters** array. Then, set the **Operator** field to either *beginswith*, *contains*, *endswith*, or = (equals)and the	**Example**: You want to pull back every contact that had a comment associated with it with the words "review later".
		
		`"SearchFilters": [`
		`  {`
		`  "TableFilterTypes": "CallComment",`
		`  "Operator": "=",`
		`  "Operand1": "review later"`

Option	How to Set It	Example
	Operand1 value to the word(s) within the comments you want to filter by.	`}` `]`

#8 Filter your export by one or more consecutive words within the comments on each score associated with various contacts.	Set the **TableFilterTypes** field to *ScoreComment* within the **SearchFilters** array. Then, set the **Operator** field to either *beginswith*, *contains*, *endswith*, or = (equals)and the **Operand1** value to the word(s) within the comments you want to filter by.	**Example**: You want to pull back every contact that had a comment on a score with the words "needs updated".
		`"SearchFilters": [`
		`{`
		`"TableFilterTypes": "ScoreComment",`
		`"Operator": "=",`
		`"Operand1": "needs updated"`
		`}`
		`]`

#9 Filter your export by the message date of new threads in a text-based contact.	Set the **TableFilterTypes** field to *SegmentStartDay* within the **SearchFilters** array. Then, set the **Operator** field to = (equals), > (greater than), or < (less than), and the **Operand1** value to a date string.
	Or, use the *between* operator and provide a date
		Example: You want to pull back every text-based contact that had a new message added to the text contact thread any day from November 1st through November 3rd.
		`"SearchFilters": [`
		`{`
		`"TableFilterTypes": "SegmentStartDay",`
		`"Operator": "between",`
		`"Operand1": "11/01/2024",`
		`"Operand2": "11/03/2024"`
		`}`
		`]`

Option	How to Set It	Example
	value for both **Operand1** and **Operand2**.	

#10 Filter your export by included metadata.	Enter each metadata item you want to filter by into its own object within the **SearchFilters** array.
	
	Then, set the **ColumnName** values to the **ColumnName** of the metadata items you want to filter by. (To obtain a list of **ColumnName** values in your Eureka system, use the Retrieve Metadata Information route.)
	
	Set **Operator** to = (equals) and **Operand1** to the metadata value you want to filter by.
		Example: You want your export to only include contacts handled by agent *John Smith* and agent *Jane Doe*.
		
		`"SearchFilters": [`
		`  {`
		`  "ColumnName": "Agent",`
		`  "Operator": "=",`
		`  "Operand1": "John Smith"`
		`  },`
		`  {`
		`  "ColumnName": "Agent",`
		`  "Operator": "=",`
		`  "Operand1": "Jane Doe"`
		`  }`
		`  ]`

#11 Filter your export by excluded metadata.	Enter each metadata item you want to filter by into its own object within the **SearchFilters** array.
	
	Then, set the **ColumnName** values to the **ColumnName** of
		Example: You want your export to include all contacts *except* those handled by agent *John Smith* or agent *Jane Doe*.
		
		`"SearchFilters": [`
		`  {`
		`  "ColumnName": "Agent",`
		`  "Operator": "!=",`

Option	How to Set It	Example

	the metadata items you want to filter by. (To obtain a list of **ColumnName** values in your Eureka system, use the Retrieve Metadata Information route.)
	
	Set **Operator** to *!=* (not equals) and **Operand1** to the metadata value you want to filter by.
		`"Operand1": "John Smith"`
		`},`
		`{`
		`"ColumnName": "Agent",`
		`"Operator": "!=",`
		`"Operand1": "Jane Doe"`
		`}`
		`]`

#12 Filter your export by contacts that fell into a specific metadata range based on the name of the range.	Enter the **ColumnName** value of the metadata item the range belongs to in the **SearchFilters** array. Then, set **Operator** to = (equals) and **Operand1** to the name of the metadata range. (To obtain a list of **ColumnName** values and metadata **Ranges** name values in your Eureka system, use the Retrieve Metadata Information route.)	**Example**: You want to export every contact that fell into the *Longest Hold Time* range of your **Hold Time** metadata item.
		
		`"SearchFilters": [`
		`{`
		`"ColumnName": "Hold Time",`
		`"Operator": "=",`
		`"Operand1": "Longest Hold Time"`
		`}`
		`]`

Option	How to Set It	Example

#13 Export data from a specific time range from the previous day.	Within the **`SearchFilters`** array, enter *ClientCaptureTime* as the **`ColumnName`** value and set **`Operator`** to *between*. Then, use *HH:mm:ss* format and enter the start time of the range you want to export as the value for **`Operand1`** and the end time of the range as the value for **`Operand2`**.
	
	Set your **`Duration`** object to **`LastNDays`** with a value of *1*.
		Example: You want to export every contact that occurred between 9 AM and 5 PM the day prior.
		
		`"Duration": {`
		`  "LastNDays": 1,`
		`  "LastNHours": null,`
		`  "TimeFrame": null,`
		`  "StartDate": null,`
		`  "EndDate": null`
		`},`
		`"SearchFilters": [`
		`  {`
		`  "ColumnName": "ClientCaptureTime",`
		`  "Operator": "between",`
		`  "Operand1": "09:00:00",`
		`  "Operand2": "17:00:00"`
		`  }`
		`  ]`

#14 Filter your export by other metadata options.	The sky (and your system configuration) is the limit here. Get creative! You can use many different combinations of metadata filters depending on your specific use-case and business needs. You can also use a combination of **`Operator`** values with one or both **`Operand`** fields to	**Example**: You want to filter your export by *audio* contacts that had *Long Hold Time* and were handled by your *Customer Service* department.
		
		`"SearchFilters": [`
		`  {`
		`  "ColumnName": "Contact Type",`
		`  "Operator": "=",`
		`  "Operand1": "call"`
		`  },`
		`  {`
		`  "ColumnName": "Department",`
		`  "Operator": "=",`
		`  "Operand1": "Customer Service"`

Option	How to Set It	Example

	achieve your desired results.
	
	Note: See the "Operator Options" section following this table for more details and examples.
		`},`
		`{`
		`"ColumnName": "Hold Time",`
		`"Operator": "=",`
		`"Operand1": "Long Hold Time"`
		`}`
		`]`

#15 Combine multiple `SearchFilters` options and use them together.	Enter as many objects within the `SearchFilters` array as you'd like to achieve the exact filtered results you are looking for.
	
	If you do not wish to use any of the other `SearchFilters` options within a given object, either remove the fields from the request or set them to *null*.
		Example: You want to filter your export by contacts that had a hit on the category with ID *126336* and were handled by agent *John Doe*.
		
		`"SearchFilters": [`
		`{`
		`"CategoryId": 126336,`
		`"Operator": "="`
		`},`
		`{`
		`"ColumnName": "Agent",`
		`"Operator": "=",`
		`"Operand1": "John Doe"`
		`}`
		`]`

Operator Options

When you filter which data you want to be included in your export, you have quite a few `Operator` filter options to choose from:

Operator Value	Operator Meaning	Example	What that Looks Like in Code
=	equals	You want to export every contact within a provided duration that has **Contact Type** equal to *chat*.	`"SearchFilters": [ { "ColumnName": "Contact Type", "Operator": "=", "Operand1": "chat", "Operand2": null } ]`
!=	does not equal	You want to export every contact within a provided duration except those that have **Contact Type** equal to *chat*, meaning your export will include all calls, video contacts, emails, etc.	`"SearchFilters": [ { "ColumnName": "Contact Type", "Operator": "!=", "Operand1": "chat", "Operand2": null } ]`
>	is greater than	You want to export every contact within a duration that has a **Word Count** greater than *50*.	`"SearchFilters": [ { "ColumnName": "Word Count", "Operator": ">", "Operand1": "50", "Operand2": null } ]`

Operator Value	Operator Meaning	Example	What that Looks Like in Code
>=	is greater than or equal to	You want to export every contact within a duration that has a **Word Count** greater than or equal to *50* words.	`"SearchFilters": [ { "ColumnName": "Word Count", "Operator": ">=", "Operand1": "50", "Operand2": null } ]`
!>	is not greater than	You want to export every contact within a duration that does not have a **Word Count** greater than *50,* meaning all contacts with 50 words or fewer.	`"SearchFilters": [ { "ColumnName": "Word Count", "Operator": "!>", "Operand1": "50", "Operand2": null } ]`
<	is less than	You want to export every contact that has a **Duration** less than *Average Duration*, meaning you want to export every contact that fell into the	`"SearchFilters": [ { "ColumnName": "Word Count", "Operator": "<", "Operand1": "Average Duration", "Operand2": null } ]`

Operator Value	Operator Meaning	Example	What that Looks Like in Code
		Short Duration and *Shortest Duration* metadata ranges.	
<=	is less than or equal to	You want to export every contact that has a **Duration** less than or equal to *Average Duration*, meaning you want to export every contact that fell into the *Average Duration*, *Long Duration*, and *Longest Duration* metadata ranges.	`"SearchFilters": [ { "ColumnName": "Word Count", "Operator": "<=", "Operand1": "Average Duration", "Operand2": null } ]`
!<	is not less than	You want to export every contact that does not have a **Duration** less than *Average Duration*,	`"SearchFilters": [ { "ColumnName": "Word Count", "Operator": "!<", "Operand1": "Average Duration", "Operand2": null } ]`

Operator Value	Operator Meaning	Example	What that Looks Like in Code
		meaning you want to export every contact that fell into the *Long Duration* and *Longest Duration* metadata ranges.	
contains	the value contains	You want to export all contacts that have a comment associated with them that contains the word "great".	`"SearchFilters": [ { "TableFilterTypes": "CallComment", "Operator": "contains", "Operand1": "great", "Operand2": null } ]`
!contains	the value does not contain	You want to export all contacts that have a comment associated with them that does not contain the word "great".	`"SearchFilters": [ { "TableFilterTypes": "CallComment", "Operator": "!contains", "Operand1": "great", "Operand2": null } ]`
between	the value is between	You want to export all contacts within the	`"SearchFilters": [ {`

Operator Value	Operator Meaning	Example	What that Looks Like in Code

		provided duration that have a **Eureka ID** (a Contact ID) between *354566* and *354576*.
		
		Note: This filter is inclusive, meaning it will also pull back the contacts with *354566* and *354576* as their IDs.
			`"ColumnName": "Eureka ID",`
			`"Operator": "between",`
			`"Operand1": "354566",`
			`"Operand2": "354576"`
			`}`
			`]`

!between	the value is not between	You want to export all contacts within the provided duration except those that have a **Eureka ID** (a Contact ID) between	`"SearchFilters": [`
			`{`
			`"ColumnName": "Eureka ID",`
			`"Operator": "!between",`
			`"Operand1": "354566",`
			`"Operand2": "354576"`
			`}`
			`]`

Operator Value	Operator Meaning	Example	What that Looks Like in Code
		354566 and *354576*. > **Note**: This filter is inclusive, meaning it will also not pull back the contacts with *354566* and *354576* as their IDs.	

beginswith	the value begins with	You want to export all contacts within the provided duration that have a comment on them that begins with the word "quality".	`"SearchFilters": [`
			`  {`
			`  "ColumnName": "Comment",`
			`  "Operator": "beginswith",`
			`  "Operand1": "quality",`
			`  "Operand2": null`
			`  }`
			`  ]`

!beginswith	the value does not begin with	You want to export all contacts	`"SearchFilters": [` `  {`

Operator Value	Operator Meaning	Example	What that Looks Like in Code

		within the provided duration that do not have a comment on them that begins with the word "quality".	`"ColumnName": "Comment",`
			`"Operator": "!beginswith",`
			`"Operand1": "quality",`
			`"Operand2": null`
			`}`
			`]`

endswith	the value ends with	You want to export all contacts that have an **Agent Name** that ends with the letters "er".	`"SearchFilters": [`
			`{`
			`"ColumnName": "Agent",`
			`"Operator": "endswith",`
			`"Operand1": "er",`
			`"Operand2": null`
			`}`
			`]`

!endswith	the value does not end with	You want to export all contacts that have an **Agent Name** that does not end with the letters "er".	`"SearchFilters": [`
			`{`
			`"ColumnName": "Agent",`
			`"Operator": "!endswith",`
			`"Operand1": "er",`
			`"Operand2": null`
			`}`
			`]`

isnull	the value is null	You want to export all contacts within the provided duration that have a **Department**	`"SearchFilters": [`
			`{`
			`"ColumnName": "Department",`
			`"Operator": "isnull",`
			`"Operand1": null,`
			`"Operand2": null`
			`}`
			`]`

Operator Value	Operator Meaning	Example	What that Looks Like in Code
		set to *null*.	
!isnull	the value is not null	You want to export all contacts within the provided duration that do not have their **Department** set to *null*.	`"SearchFilters": [ { "ColumnName": "Department", "Operator": "!isnull", "Operand1": null, "Operand2": null } ]`

And remember, you can use many different combinations of filters depending on your specific use-case and business needs, so get creative and see what works best for you!

Appendix H

Retrieve Content `SearchFilters` Values

You can use a variety of content types to filter the data that will be returned in a given export. Aside from the huge variety of metadata filters available to you, you can also filter your exports by one or more of the following:

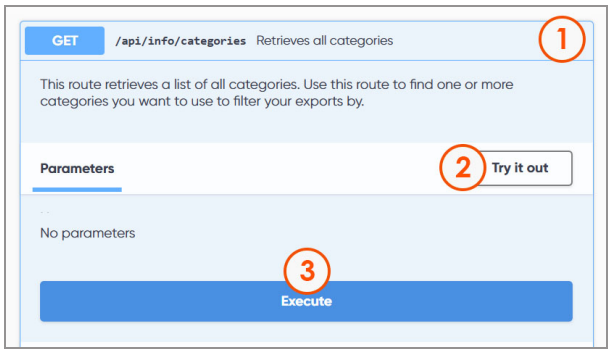
- Categories
- Tags
- Sections
- Scores
- Alerts

To filter a given export by any of these content types, you need to know the ID value of the category, tag, score, etc. that you want to filter by. To make things easier, we've included informational routes in the Bulk Export API to retrieve these ID values as well as other information about each piece of content.

To retrieve an ID value you can use in the `SearchFilters` array on either the POST or PUT export route:

> **Important**: We're going to use the **GET /api/info/categories** route in our example, but know that each of the other content routes work the same way.

Using the Interactive Documentation



1. Press the **GET /api/info/categories** route to expand it.
2. Press **Try it out**.
3. Press **Execute**.

Using a Third-Party Application

1. Create a **GET** request with the **/api/info/categories** route and the CallMiner Bulk Export API URL. Or, choose one of the other content routes:
   - **/api/info/tags**
   - **/api/info/sections**
   - **/api/info/scores**
   - **/api/info/alerts**
2. Set the following request headers:
   - **Authorization**: Set the value to the word *Bearer* followed by a space and then the authorization token. For example, *Bearer abc1234*.
   - **Content-Type**: Set the value to *application/json; charset=utf-8*.

   For example:

   ```
   GET /api/info/categories
   Host: Bulk_Export_API_URL
   Authorization: Bearer abc1234
   Content-Type: application/json;
   charset=utf-8
   ```
3. Send the request.

The Response

> **Important**: If an error occurs, an HTTP status code in the `400` or `500` range is returned. Refer to the message provided with the error and HTTP Status Codes to troubleshoot what went wrong, correct the request, and try again.

If the request is successful, you will receive a `200` HTTP status code and a response body that contains the ID value of each piece of content (in our example, each category and its associated `CategoryId` value) in your Eureka system.

Using any of the available content routes, note the `CategoryId`, `TagId`, `SectionId`, `ScoreId`, or `AlertId` value(s) of each content item you want to use in the `SearchFilters` array to filter your exported data and then return to either the Create a Job to Export Contact Data section or the Update a Recurring Export Job section to continue configuring your export job.

> **Important**: While the equivalent routes in the CallMiner Data API may return dates related to the various content types in different time zone formats, all of the datetime field values in the Bulk Export API are provided in UTC.

For more information on the other fields returned with each content type, see below:

Categories

The **GET /api/info/categories** route returns the following for each category in your system:

```
{
   "Id": 3454355,
   "Name": "Emotions",
   "Description": "Category tracking emotions.",
   "SectionId": 1258,
   "Threshold": 1,
   "Created": "2025-03-07T12:33:58.133Z",
   "CreatedBy": "user1@callminer.com",
   "LastModified": "2025-03-07T18:38:58.133Z",
   "LastModifiedBy": "user2@callminer.com",
   "Version": 1,
   "Components": [
      {
```

```
            "Id": 5756453,
            "Name": "Anger",
            "Weight": 1,
            "CountMultiple": true,
            "Status": "Normal"
        }
    ]
}
```

- **Id** (integer): The unique identifier of the category. If you'd like to use this category as a filter on one of your export jobs, take this `Id` value and enter it as a `CategoryId` value in an object in the `SearchFilters` array on either the Create a Job to Export Contact Data or Update a Recurring Export Job routes.
- **Name** (string): The name of the category.
- **Description** (string): The description for the category.
- **SectionId** (integer): The unique identifier for the section the category belongs to.
- **Threshold** (number): The threshold that a contact must surpass in order to be counted as having a hit for the category. An individual category component can be set to only count for one hit per contact, regardless if the component appeared multiple times in a contact, or to count every time the component hit for a contact. Each component hit is then multiplied by the overall `Weight` for the component. If the summed `Weight` values for each category component hit exceed the `Threshold` value set for the category, the contact has a category hit for that category.
- **Created** (datetime string): The date and time that the category was created (in UTC).
- **CreatedBy** (string): The username of the person who created the category.
- **LastModified** (datetime string): The date and time the category was last modified (in UTC).
- **LastModifiedBy** (string): The username of the person who last modified the category.
- **Version** (integer): The version number of the category.
- **Components** (array): The list of components that belong to the category.
    - **Id** (integer): The unique identifier of the component.
    - **Name** (string): The name of the component.
    - **Weight** (number): The multiplier value used to calculate a category hit based on the number of components that the contact hit on. An individual component can be set to only count for

one hit per contact, regardless if the component appeared multiple times in a contact, or to count every time the component hit for a contact. Each component hit is then multiplied by the overall `Weight` for the component. If the summed `Weight` values for each component hit exceed the `Threshold` value set for the category, the contact has a category hit for that category.

- **CountMultiple** (boolean): Indicates if the search component is counted once or multiple times during category hit calculation. If *true*, the component can be counted multiple times if the component appears in a given contact more than once. If *false*, the component will only count for one hit, even if there are multiple instances of the component occurring in a single contact.
- **Status** (enum): The status of the component. *Off* indicates that the component is disabled. *Normal* indicates it is enabled. *ManualIntraCall* indicates that the component can be applied to a snippet of the contact manually. *ManualEntireCall* indicates that the component can be applied to an entire contact manually.

Tags

The **GET /api/info/tags** route returns the following for each tag in your system:

```
{
    "Id": 3454355,
    "Name": "Review Later"
}
```

- **Id** (integer): The unique identifier of the tag. If you'd like to use this tag as a filter on one of your export jobs, take this `Id` value and enter it as a `TagId` value in an object in the `SearchFilters` array on either the Create a Job to Export Contact Data or Update a Recurring Export Job routes.
- **Name** (string): The name of the tag.

Sections

The **GET /api/info/sections** route returns the following for each section in your system:

```
{
    "Id": 3454355,
    "Name": "Emotion Categories",
    "IsCategory": true
}
```

- **Id** (integer): The unique identifier of the section. If you'd like to use this section as a filter on one of your export jobs, take this `Id` value and enter it as a `SectionId` value in an object in the `SearchFilters` array on either the Create a Job to Export Contact Data or Update a Recurring Export Job routes.
- **Name** (string): The name of the section.
- **IsCategory** (boolean): Indicates if the section contains categories. If *true*, the section contains categories. If *false*, the section contains tags.

Scores

The **GET /api/info/scores** route returns the following for each score in your system:

```
{
   "Id": 3454355,
   "Name": "Agent Quality",
   "Description": "Scores a contact center agent's performance.",
   "CurrentlyActive": true,
   "Created": "2024-07-07T15:01:18.763Z",
   "CreatedBy": "user1@callminer.com",
   "LastModified": "2025-03-07T13:01:11.763Z",
   "LastModifiedBy": "user2@callminer.com",
   "Version": 2,
   "Ranges": [
      {
         "RangeNumber": 1,
         "RangeMin": null,
         "RangeMax": 50,
         "RangeFriendlyName": "Bad",
         "RangeColor": "#FF0000",
         "Target": false
      },
      {
         "RangeNumber": 2,
         "RangeMin": 51,
         "RangeMax": 100,
         "RangeFriendlyName": "Good",
         "RangeColor": "#008000",
         "Target": true
      }
   ],
   "ScoreIndicators": [
```

```
        {
            "Id": 7474636,
            "Name": "Compliments",
            "FullName": "Agent Quality.Compliments",
            "Description": "Scores agent on use of compliments.",
            "LastModified": "2025-03-07T13:01:11.763Z",
            "LastModifiedBy": "user3@callminer.com",
            "CurrentlyActive": true,
            "Ranges": [
                {
                    "RangeNumber": 1,
                    "RangeMin": null,
                    "RangeMax": 50,
                    "RangeFriendlyName": "Bad",
                    "RangeColor": "#FF0000",
                    "Target": false
                },
                {
                    "RangeNumber": 2,
                    "RangeMin": 51,
                    "RangeMax": 100,
                    "RangeFriendlyName": "Good",
                    "RangeColor": "#008000",
                    "Target": true
                }
            ]
        }
    ]
}
```

- **Id** (integer): The unique identifier of the score. If you'd like to use this score as a filter on one of your export jobs, take this `Id` value and enter it as a `ScoreId` value in an object in the `SearchFilters` array on either the Create a Job to Export Contact Data or Update a Recurring Export Job routes.
- **Name** (string): The name of the score.
- **Description** (string): The description of the score.
- **CurrentlyActive** (boolean): Indicates if the score is enabled or disabled. If *true*, the score is enabled. If *false*, the score is disabled.
- **Created** (datetime string): The date and time that the score was created (in UTC).
- **CreatedBy** (string): The username of the person who created the score.

- **LastModified** (datetime string): The date and time that the score was last modified (in UTC).
- **LastModifiedBy** (string): The username of the person who last modified the score.
- **Version** (integer): The version number of the score.
- **Ranges** (array): The list of ranges for the score. For example, you might have three ranges that contain different ratings for the **Agent Politeness** score indicator. The `Weak` range for the indicator could hold any contacts that score *0-50*, the `Acceptable` range might hold contacts that score *50-75*, and the `Good` range might hold contacts that score *75-100*.
    - **RangeNumber** (integer): The number of the score range out of the total number of score ranges for the score.
    - **RangeMin** (number): The minimum value for the score range.
    - **RangeMax** (number): The maximum value for the score range.
    - **RangeFriendlyName** (string): The friendly name for the score range. For example, a range could be labeled *Weak*, *Acceptable*, or *Good*.

      > **Note**: You can use this value in and object in the `SearchFilters` array `Operand1` field value if you want to filter your exports by this score range. See the Create a Job to Export Contact Data and Update a Recurring Export Job `SearchFilters` sections for more information.

    - **RangeColor** (string): The color assigned to the score range. The color will appear within Analyze to easily indicate what value range a contact scored in.
    - **Target** (boolean): Indicates if the score range has been designated as the target range. For example, if you have a score for **Profanity**, the target range for that score might be *0-2*. If *true*, the score range has been designated as the target range. If *false*, the score range hasn't been designated as the target range.
- **ScoreIndicators** (array): The list of indicators that belong to the score.
    - **Id** (integer): The unique identifier of the indicator.
    - **Name** (string): The name of the indicator.
    - **FullName** (string): The full name of the indicator, which includes the name of the score the indicator belongs to.
    - **Description** (string): The description of the score indicator.
    - **LastModified** (string): The date and time that the score indicator was last modified (in UTC).

- **LastModifiedBy** (string): The username of the person who last modified the score indicator.
- **CurrentlyActive** (boolean): Indicates if the score indicator is enabled or disabled. If *true*, the score indicator is enabled. If *false*, the score indicator is disabled.
- **Ranges** (array): The list of ranges for the score indicator. For example, you might have three ranges that contain different ratings for the **Agent Politeness** score indicator. The `Weak` range for the indicator could hold any contacts that score *0-50*, the `Acceptable` range might hold contacts that score *50-75*, and the `Good` range might hold contacts that score *75-100*.
    - **RangeNumber** (integer): The number of the score indicator range out of the total number of ranges for the score indicators.
    - **RangeMin** (number): The minimum value for the score indicator range.
    - **RangeMax** (number): The maximum value for the score indicator range.
    - **RangeFriendlyName** (string): The friendly name for the score indicator range. For example, a range could be labeled *Weak*, *Acceptable*, or *Good*.
    - **RangeColor** (string): The color assigned to the score indicator range. The color will appear within Analyze to easily indicate what value range a contact scored in.
    - **Target** (boolean): Indicates if the score indicator range has been designated as the target range. For example, if you have a score for **Profanity**, the target range for that score might be *0-2*. If *true*, the score range has been designated as the target range. If *false*, the score range hasn't been designated as the target range.

Alerts

The **GET /api/info/alerts** route returns the following for each alert in your system:

```
{
    "Id": 3454355,
    "Name": "Required Greeting"
}
```

- **Id** (integer): The unique identifier of the alert. If you'd like to use this alert as a filter on one of your export jobs, take this `Id` value and enter it as an `AlertId` value in an object in the `SearchFilters` array on either the Create a Job to Export Contact Data or Update a Recurring Export Job routes.
- **Name** (string): The name of the alert.

Appendix I

Quartz Cron Basics

When creating an export job with a recurring delivery schedule, you must create a Quartz cron expression for your **Schedule** value.

Below, you'll find details and examples of how to build your Quartz cron expression.

The cron pattern consists of 6 or 7 space-separated datetime fields in the following order:

```
<second> <minute> <hour> <day of the month> <month> <day of the week> <year>
```

Order	Field	Required	Allowed Values	Allowed Special Characters	Examples
1	second	✓	*0-59*	, - * /	For the expression `30 0 0 * * ?`, the *30* combined with the other values indicates that the export will deliver your data 30 seconds after midnight every day.
2	minute	✓	*0-59*	, - * /	For the expression `30 10 0 * * ?`, the *10* combined with the other values indicates that the export will deliver your data 10 minutes and 30 seconds after midnight every day.
3	hour	✓	*0-23*	, - * /	For the expression `30 10 8 * * ?`, the *8* combined with the other values indicates that the export will deliver your data 8 hours, 10 minutes, and 30 seconds after midnight every day.
4	day of the month	✓	*1-31*	, - * ? / L W	For the expression `30 10 8 5 * ?`, the *5* combined with the other values indicates that the export will deliver your data 8 hours, 10 minutes, and 30 seconds after midnight on

Order	Field	Required	Allowed Values	Allowed Special Characters	Examples
					the 5th day of every month.
5	month	✓	*1-12* or *JAN-DEC*	, - * /	For the expression `30 10 8 5 12 ?`, the *12* combined with the other values indicates that the export will deliver your data 8 hours, 10 minutes, and 30 seconds after midnight on the 5th day of December, every year.
6	day of the week	✓	*1-7* or *SUN-SAT*	, - * ? / L #	For the expression `30 10 8 ? 12 2`, the *2* combined with the other values indicates that the export will deliver your data 8 hours, 10 minutes, and 30 seconds after midnight every Monday in December, every year.
7	year	✕	*1970-2099* or leave it empty	, - * /	For the expression `30 10 8 ? 12 1 2025`, the *2025* combined with the other values indicates that the export will deliver your data 8 hours, 10 minutes, and 30 seconds after midnight every Monday in December in the year 2025.

Important: Any times you enter for your Quartz cron schedule should be in UTC, and the most frequent delivery schedule you can set is exporting your data every 1 hour.

Important: You may only enter a `<day of the month>` or a `<day of the week>` value in your expression, not both. Place a ? into the value for whichever field you are not using. See the section below for more information.

Special Character Meanings

Special Character	How it Works	Example
*	This indicates you want to use every value of a particular field.	**Example**: *0 0 * ? * * ** Placing a * in the `<hour>`, `<month>`, `<day of the week>`, and `<year>` values indicates you want your export to run on every hour of the day, regardless of the month, day of the week, or year.
?	This indicates that a particular value is not used for a given expression.	**Example**: *0 30 * * * ?* You would place a ? in the `<day of the week>` position because the * already indicates that you want your export to run 30 minutes past every hour, every day. Because it's running every day, you cannot also specify a `<day of the week>` value. You must choose one or the other and use the ? for the field that you are not using.
-	This indicates a range is being used.	**Example**: *0 30 * ? * MON-FRI* Placing a - in between two values in the `<day of the week>` field indicates you want your export to run on every day within that range. The example above will export data 30 minutes past every hour, Monday through Friday.
,	This indicates that you are entering additional values for a given cron field.	**Example**: *0 30 * ? * MON,FRI* Placing a , between two `<day of the week>` values indicates you want your export to run on both days. The example above will export data 30 minutes past every hour on Mondays and Fridays only.
/	This indicates that the value you are entering	**Example**: *0 0 0 1/3 * ?*

Special Character	How it Works	Example
	is an increment value.	Placing a / in the `<day of the month>` field between two values will export your data on every 3rd day, starting on the 1st day of the month. This will trigger the job at midnight on the 1st, 4th, 7th, 10th, and so on throughout the month.
L	Applies to either the `<day of the month>` or the `<day of the week>` field. Using it indicates the last day of the week or month.	**Example**: *0 0 0 L * ? ** Placing an *L* in the `<day of the month>` field indicates that your export should deliver your data on the last day of every month, regardless if the last day falls on the 28th, 29th, 30th, or 31st. **Example**: *0 0 0 ? * L ** If you place an *L* in the `<day of the week>` field by itself, your export will deliver your data every Saturday. **Example**: *0 0 0 ? * 2L ** If you place an *L* in the `<day of the week>` field next to the number of a given day (1-7 or Sunday through Saturday respectively), your job will export your data on the last day (in our example Monday) of the month every month. **Note**: Do not use lists or ranges of values when using the *L* operator; if you do, you may get unexpected results with your export schedule.
W	Only applies to the `<day of the month>` field and is used to indicate the nearest given weekday (Monday- Friday).	**Example**: *0 0 0 18W * ? ** Placing a *W* next to a `<day of the month>` field will export your data on the nearest weekday to your provided date if your date falls on the weekend. So, in our example, if the 18th falls on a Monday, the data will be delivered that day, but if the 18th falls

Special Character	How it Works	Example
		on a Saturday, the data will be delivered on Friday the 17th. If you enter *1W* as your value and the 1st happens to be on a Saturday, your data will be delivered on Monday the 3rd instead as this field will respect the provided month boundary. **Note**: Do not use lists or ranges of values when using the *W* operator in the `<day of the month>` field. **Pro-tip**: Combine the *L* and *W* values together to indicate you want your export to run on the last weekday of every month. For example: *0 0 0 LW * ? ****
#	This is used to indicate an *nth* day of the month or the *nth* day of the week.	**Example**: *0 30 11 ? * 6#2* Placing the #2 next to the number 6 in the `<day of the week>` field indicates that you want your data exported on the second Friday of every month (since the 6 refers to Friday; the digits 1-7 refer to Sunday through Saturday respectively).

More Examples

Quartz Cron Expression	When the Export Runs
0 30 * * * ?	At the 30-minute mark of every hour every day.
0 0 13-15 * * ? 2025	At 1 and 3 P.M. every day during the year 2025.
0 0 6,18 ? * *	At 6:00 A.M. and 6:00 P.M. every day.
0 0 9-17 * * MON-FRI	At every hour from 9 A.M. to 5 P.M. Monday through Friday.
0 59 23 L * ?	On the last day of every month at 11:59 P.M.

> **Note**: The names of months and the days of the week are not case sensitive in a Quartz cron expression. For example, you may enter *FRI* or *fri* (Friday) into your expression or *JAN* or *jan* (January) and it will work.

> **Pro-tip**: Use an online Quartz cron expression builder to easily create your `Schedule` value.

Appendix J

Best Practices and FAQs

The following are groupings of frequently asked questions and their responses.

My export is too large and it's affecting system performance...

Q: What can we do to avoid this?

A: Modify your job so it exports data in smaller chunks or batches more frequently.

Example A: If your current job exports data from the previous day every time using the **`LastNDays`** option, but the amount of data in that single export is too large, consider using the **`LastNHours`** option instead and setting a more frequent delivery **`Schedule`**. For example:

```
"Duration": {
      "LastNDays": null,
      "LastNHours": 4,
      "TimeFrame": null,
      "StartDate": null,
      "EndDate": null,
   },
"Schedule": "0 */4 * * *"
```

This **`Duration`** and **`Schedule`** setting would export your data from the last 4 hours, every 4 hours, every day. You'd receive 8 exports a day from this job with smaller chunks of data, instead of one giant export with all of the data from the last 24 hours.

> **Note**: The most frequent delivery **`Schedule`** you can set is exporting your data every 1 hour and the minimum you can set **`LastNHours`** to is *1* (which exports data from the previous hour, every hour).

Example B: If your current export job exports data from a large date range (e.g. the last 7 days) using the `LastNDays` option, but the amount of data in that single export is too large, consider using a smaller `LastNDays` value instead (or one of the `LastNHours` options) and setting a more frequent delivery `Schedule`. For example:

```
"Duration": {
      "LastNDays": 1,
      "LastNHours": null,
      "TimeFrame": null,
      "StartDate": null,
      "EndDate": null,
   },
"Schedule": "0 1 * * *"
```

This `Duration` and `Schedule` setting would export your data from the previous day, every day at 1AM, thus making the amount of data in your export smaller.

Handling updated versions of contacts...

Q: What should we do when we get updated versions of contacts?

A: It depends on what you need for your particular use case! If you don't need to keep track of past changes, you can simply overwrite the old contact data with the new version.

Note: You receive duplicated contacts in your exports when you have your `SearchMode` set to either *Updated* or *NewAndUpdated* and retroprocessing occurs. If you don't need to keep track of updates to your contacts post-retroprocessing, use the *ClientCaptureDate* or *CreateDate* `SearchMode` option in your exports to avoid receiving duplicated contacts entirely.

But, if you want to keep a history or see how the data changes over time, we recommend that you include the `Contacts` data type in your exports and use versioning. Keep a record of each contact update using the `LastModified` date and time value of each contact provided in the `Contacts` CSV file in each export.

Combining all data into one table...

Q: Can we put all the scores, categories, and events into a single "CONTACT" table with everything in one row?

A: You can, but it usually ends up being more complicated and less efficient. Because there are so many categories and scores, and most don't apply to every contact, you'd end up with a lot of empty or "zero" spots in your table.

For example, if you have 100 categories, only about 10% may apply to any given contact, meaning the majority of fields in a flattened row would be empty or zero—resulting in a very large table that might be hard to work with.

Instead, we suggest keeping the data organized like it comes from the export—that means the information is split into related tables. This setup works well with tools like Power BI and Tableau that can easily combine the data when needed.

Also, it's important to avoid flattening contact transcripts into the table. Transcripts are long text fields, and adding them would make the table even wider and slower to work with. Keeping transcripts separate helps your data stay clean and your analyses fast.

Bringing data into Snowflake or other data storage systems...

Q: How do we get the data into Snowflake or similar databases?

A: Snowflake has tools that let you automatically import your exported files, making it pretty easy to add new data.

Updating existing data is a little trickier, though. If something changes about a contact's data, the old data generally needs to be deleted—often via a cascade delete starting from the CONTACT table—and then replaced with the updated records to maintain integrity.

While Snowflake provides a MERGE function that acts like an UPSERT for updates, it doesn't fully address deletions for category, score, etc. data. For example, if a category was associated with a contact previously but is no longer relevant, the MERGE will not remove this outdated data. Therefore, a delete-and-replace workflow is recommended to ensure your Snowflake (or whichever storage system you're using) data accurately reflects the current state.

Scaling with large volumes of data...

Q: How does the system scale if we have a large amount of data?

A: To efficiently handle large volumes of data, we recommend using rolling hourly or daily exports.

Example A:

```
"Duration": {
      "LastNDays": 1,
      "LastNHours": null,
      "TimeFrame": null,
      "StartDate": null,
      "EndDate": null,
   },
"Schedule": "0 1 * * *"
```

This `Duration` and `Schedule` setting would export your data from the previous day, every day at 1AM, thus making it easier to scale your data.

Example B:

```
"Duration": {
      "LastNDays": null,
      "LastNHours": 4,
      "TimeFrame": null,
      "StartDate": null,
      "EndDate": null,
   },
"Schedule": "0 */4 * * *"
```

This `Duration` and `Schedule` setting would export your data from the last 4 hours, every 4 hours, every day. You'd receive 8 exports a day from this job with smaller chunks of data, instead of one giant export with all of the data from the last 24 hours.

Example C: You can even create a job that exports your data from the previous hour every hour every day, like so:

```
"Duration": {
      "LastNDays": null,
      "LastNHours": 1,
      "TimeFrame": null,
      "StartDate": null,
      "EndDate": null
},
"Schedule": "0 * * * *"
```

Note: This is the most frequent delivery schedule you can set for your exports.

Handling duplicate contact exports caused by retroprocessing...

Q: Why are we seeing duplicate contacts in our exports, and how should we handle them?

A: When contacts are retroprocessed (updated after the original export), those contacts become candidates for export again—even if no actual data within the contact has changed. Simply "touching" a contact during retroprocessing marks it as updated, so it will be included in the next export. This can result in duplicate records appearing in your exported data.

There are a few ways you can reduce duplicates in your exports:

1. **Limit the time span (retroactive date) of each retroprocessing run** to avoid going back further than needed. This helps minimize unnecessary updates.

   Example: Instead of retroprocessing data for the last 30 days, limit it to the last 7 days (or fewer) to create the fewest number of changes possible.

2. **Don't make changes to a category and then have the system retroprocess contacts unless absolutely necessary.**

When you update a category and choose a retroactive date, all contacts will be retroprocessed within the specified date range, which can cause a large number of contacts to be included in the next export(s).

> **Example**: If you make a change to your *Empathy* category and set it to retroprocess contacts for the last 7 days, every contact that occurred in the system in the last week will be retroprocessed to see if it has a hit for the category with the new changes. The Bulk Export system will see that all of these contacts have updates and will export them again.

> **Note**: Choosing a retroactive date when creating or updating a category is optional. If you do not set a retroprocess date, the system will only apply the category changes to new contacts, and you will not create duplicated contacts in your exports.
>
>
>

3. **Don't create a new category and then tell the system to retroprocess contacts with it unless absolutely necessary.**

   This scenario is very similar to updating an existing category above and causes issues for the same reason.

> **Example**: If you make a new *Complaints* category and set it to retroprocess contacts for the last 7 days, every contact that occurred in the system in the last week will be retroprocessed to see if it had a hit for your new category. The Bulk Export system will see that all of these contacts have updates, and will export them again.

By applying these methods, you can keep your export data efficient and reduce duplicates.

Appendix K

HTTP Status Codes

HTTP status codes are standard response codes given by web servers. They indicate whether or not a request has been successfully completed. Below is a list of common HTTP status codes that you may receive when you use the Bulk Export API:

Code	Reason
200	OK -- The GET, PUT, PATCH, or DELETE request was successful. Can also be used for a POST request that doesn't result in a creation.
201	Created -- The POST or PUT request was successful and a resource was created or updated as a result.
401	Unauthorized -- The request requires authentication. If you get this error, double check that you're logged in to the API (when using the interactive documentation) or that you've included the correct authorization token without typos (when using third-party applications). If the error persists, contact support@callminer.com.
429	Rate Limit Exceeded -- The user has sent too many requests in a given amount of time.
500	Internal Server Error -- The server encountered an unexpected condition, which prevented it from fulfilling the request. If the error persists, contact support@callminer.com.
504	Gateway Timeout -- The server, while acting as a gateway or proxy, did not receive a timely response from the upstream server or some other auxiliary server it needed to access to complete the request. If the error persists, contact support@callminer.com.

For a complete list of HTTP status codes, see https://www.rfc-editor.org/status-codes.

Appendix L

Troubleshooting

If you received the following email from the CallMiner Bulk Export system:

```
Email Subject Line: CallMiner Bulk Export Request Failure - [Name of Export]

Email Body:

The CallMiner Bulk Export system could not export the data associated with the IDs
listed at the bottom of this email.

If this is a brand new export job, please try creating the export again using the POST
/api/export/job route. If the issue persists, please contact CallMiner support and note
the provided Export Job ID, Export History Request ID, and Correlation ID.

If you've received this error on an existing export job that has been successfully
delivering your data, see the 'Troubleshooting' section of the CallMiner Bulk Export
API User Guide. If the issue persists, please contact CallMiner support and note the
provided Export Job ID, Export History Request ID, and Correlation ID.

Export Job ID: [Export Job ID]
Export History Request ID: [Export Job History Request ID]
Correlation ID: [Error Message ID]
```

To troubleshoot the issue, go to the applicable situation below and follow the steps there:

Received Error When Creating a New Export Job 132

Received Error on an Existing Recurring Export Job 132

Received Error When Creating a New Export Job

If you received this error after creating a new job:

1. Try creating the export again using the **POST /api/export/job** route. (For steps on how to do so, see Create a Job to Export Contact Data.)
2. If the issue persists, please contact support@callminer.com.

> **Important**: When you email CallMiner support, be sure to include the `Export Job ID`, `Export History Request ID`, and the `Correlation ID` you received in the email.

Received Error on an Existing Recurring Export Job

If you received this error on an export job that has been successfully delivering your data:

1. When you receive this error, the Bulk Export system has tried to export the data multiple times but it has exceeded its retry limit. Typically, the issue will resolve itself and your next scheduled data export will succeed and deliver the data as expected. But how do you get the data from the export that failed? To do so:
   a. Go to the **GET /api/export/history** route on the CallMiner Bulk Export API.
      i. Enter the Export Job ID into the `Id` parameter.
      ii. Execute the request.
      iii. Use the Export History Request ID you received in the email message to find the specific export that failed in the response. The `Id` value in the response will match the Export History Request ID you received, and the `Status` value of the failed export will say *Failed*.
      iv. Note all of the settings that the failed job used, including the specific `Duration` that the data should have been exported from, the `Schedule` that the job was supposed to use to export the data, and the `JobCompletionTime` indicating the date and time when the job ran.
   b. Go to the **GET /api/export/job/{id}** route.

   i. Enter the Export Job ID into the `Id` parameter.
   ii. Execute the request.
   iii. Copy the entire response, which contains all of the job settings.

c. Go to the **POST /api/export/job** route.
   i. Paste the job settings you copied from the **GET /api/export/job/{id}** route into the body of this request.
   ii. Modify the `Duration` settings in the request so the job will export the same data that the original job should've exported.

Example: If you had your original job set to deliver your contact data every 7 days (on every Monday morning), your original **Duration** object might look something like this:

```
"Duration": {
      "LastNDays": 7,
      "TimeFrame": null,
      "StartDate": null,
      "EndDate": null
   },
```

And your Schedule, which tells the job to run every Monday morning at 12:01 A.M., might look something like this:

```
"Schedule": "0 1 0 ? * MON *"
```

Based on the failed export's **Duration**, **Schedule**, and **JobCompletionTime**, determine the date and time range that the data should have been exported from.

Now, within the job you are currently creating, modify the **Duration** settings so this new job will pull the same data that the failed job was supposed to deliver. Set the **TimeFrame** value to *Custom* and use the **StartDate** and **EndDate** fields to do so.

For example, if the week of data the original export failed to deliver was data from August 28th - September 1st, you might set your **Duration** to something like the following:

```
"Duration": {
      "LastNDays": null,
      "TimeFrame": "Custom",
      "StartDate": "2023-08-28T00:01:00Z",
      "EndDate": "2023-09-01T00:01:00Z"
   },
```

> This will export the data from Monday, August 28th at 12:01 A.M. to Monday, September 1st at 12:01 A.M.; it will exactly replace the week of data that the original job failed to deliver.

iii. Remove the `Schedule` field from the request. (This will make the job run one time, rather than making a duplicate recurring job, and it will deliver the data that you missed from the one failed export.)

iv. Execute the request.

v. If an error occurs, an HTTP status code in the `400` or `500` range is returned. Refer to the message provided with the error and HTTP Status Codes to troubleshoot what went wrong, correct the request, and try again.

If the request is successful, you will receive a `201` HTTP status code and a response body that contains all of the information you sent in with the request.

d. Wait for the Bulk Export system to deliver the requested data to your chosen `NotificationMethod` or `StorageTargetName` if using a third-party storage target.

e. (Optional) (Not applicable if using a third-party storage target) If you do not want to wait for the data to be delivered, you can use our **GET /api/export/history** route to poll for the `Status` of your export. To do so:

i. Take the `Id` of the export job you received in this **POST /api/export/job** response and enter it into the `Id` parameter on the **GET /api/export/history** route.

ii. Execute the route and look for the `Status` value in the response.

2. If your one-time replacement export fails, or if your recurring export job fails to deliver data again, please contact support@callminer.com.

> **Important**: When you email CallMiner support, be sure to include the `Export Job ID`, `Export History Request ID`, and the `Correlation ID` you received in the email.

Appendix M

Migration Guide: Data API to Bulk Export API

It's officially time to start moving your exports over from the Data API to the CallMiner Bulk Export API so you can take advantage of all of the amazing and expanded features the Bulk Export API has to offer.

To make the transition process easier, we've put together this migration guide. Simply navigate to the applicable use case section, find the Data API recipe you are using now in the left-hand column, and follow the provided steps to switch your export over to the Bulk Export system.

Use Cases for Getting Information Out of Your Eureka System 136

Use Cases for Exporting Data Out of Your Eureka System 139

> **Note**: To make things easier, this Migration Guide also exists as a separate document so you can have it and the Bulk Export User Guide sections mentioned in it open at the same time. If you don't have both documents, please contact your Customer Success Director.

Use Cases for Getting Information Out of Your Eureka System

Sometimes, you just want to know what categories or scores exist in your system. With the Bulk Export API, we've made a simple-to-use **Info** controller where you can easily access this content.

Deprecated Data API Cookbook Recipe	Use this Bulk Export API Route Instead	How to Use It
Getting a List of All Metadata Fields for a Tenant `GET /api/v2/metadata`	**`GET /api/info/callmetadataconfig`**	1. Go to Retrieve Metadata Information. 2. Follow the steps outlined in that section.
Getting a List of All Categories in Your Eureka System `GET /api/v2/categories`	**`GET /api/info/categories`**	1. Go to Retrieve Content SearchFilters Values. 2. Follow the steps outlined

Deprecated Data API Cookbook Recipe	Use this Bulk Export API Route Instead	How to Use It
		in that section using the **GET /api/info/categories** route.
Getting a List of All Search Components for a Saved Category `GET /api/v2/categories/{id}/components`	**GET /api/info/categories**	1. Go to Retrieve Content SearchFilters Values. 2. Follow the steps outlined in that section using the **GET /api/info/categories** route. 3. In the response, search for the ID or name of the category you are looking for. 4. All associated search components will appear within the **Components** array under the category.
Getting a List of All Scores in Your Eureka System `GET /api/v2/scores`	**GET /api/info/scores**	1. Go to Retrieve Content SearchFilters Values. 2. Follow the steps outlined in that section using the **GET /api/info/scores** route.
Getting a List of All Score Indicators for a Specific Score `GET /api/v2/scores/{scoreId}/scorecomponents`	**GET /api/info/scores**	1. Go to Retrieve Content SearchFilters Values. 2. Follow the steps outlined in that section using the **GET /api/info/scores** route.

Deprecated Data API Cookbook Recipe	Use this Bulk Export API Route Instead	How to Use It
		3. In the response, search for the ID or name of the score you are looking for. 4. All associated score indicators will appear within the **`ScoreIndicators`** array under the score.
Getting a List of All Public Tags in Your Eureka System `GET /api/v2/tags`	**`GET /api/info/tags`**	1. Go to Retrieve Content SearchFilters Values. 2. Follow the steps outlined in that section using the **`GET /api/info/tags`** route.

Important: With the CallMiner Bulk Export API, you can get information on even more types of content in your Eureka system. You can retrieve information on all of your sections and alerts as well. To do so:

1. Go to Retrieve Content SearchFilters Values.
2. Follow the steps outlined in that section using either the **`GET /api/info/sections`** or the **`GET /api/info/alerts`** route.

Use Cases for Exporting Data Out of Your Eureka System

If you're already familiar with the CallMiner Data API or the *CallMiner Data API Cookbook*, you'll find that we've taken some of our most commonly used bulk export use cases and made it even easier to export the contact data you're most interested in from your Eureka system. What required multiple Data API routes to accomplish can now be done using one simple and easy-to-use Bulk Export API route:

`POST /api/export/job`

We've also made how you export your data more dynamic. Instead of manually exporting the data each time as is required with the Data API, the Bulk Export API allows data exports to be set up on a recurring schedule. Set it and forget it, and all of your contact data will be delivered in CSV format to your email, web-hook, or third-party storage target on whatever schedule works best for you.

For steps on how to migrate your exports from the Data API to the Bulk Export API, see the following:

Deprecated Data API Cookbook Recipe	How to Use the `POST /api/export/job` Route to Do the Same Thing
Exporting Contact Data by Specific Metadata Parameter `GET /api/v2/export/metasearch`	1. Go to Create a Job to Export Contact Data. 2. Follow the steps outlined in that section to start creating a request. 3. Find the **`SearchFilters`** array in your request. 4. Within this array, you will include one object for each metadata item you want to filter by. 5. Within each metadata object, enter the **`ColumnName`** for the metadata item, set **`Operator`** to = (equals), and **`Operand1`** to the value for the metadata item. 6. Either remove the other fields that you aren't using from the object or set their value to *null*.

Deprecated Data API Cookbook Recipe	How to Use the `POST /api/export/job` Route to Do the Same Thing

	Example: If you wanted to export all contacts that were handled by your `Agent` *John Smith* in your *New York* `Location`, the **`SearchFilters`** array in your request might look something like this:
	`"SearchFilters": [`
	`  {`
	`  "ColumnName": "Agent",`
	`  "Operator": "=",`
	`  "Operand1": "John Smith"`
	`  },`
	`  {`
	`  "ColumnName": "Location",`
	`  "Operator": "=",`
	`  "Operand1": "New York"`
	`  }`
	`]`
	
	7. Continue following the steps in the *Bulk Export API User Guide* section and send the request.

Exporting Contact Data by Date `GET /api/v2/export/datesearch`	1. Go to Create a Job to Export Contact Data. 2. Follow the steps outlined in that section to start creating a request. 3. Find the **`Duration`** object in your request. 4. Within that object, set **`LastNDays`** and **`LastNHours`** to *null*, **`Timeframe`** to *Custom*, and enter a valid date for **`StartDate`** and **`EndDate`**.

Deprecated Data API Cookbook Recipe	How to Use the `POST /api/export/job` Route to Do the Same Thing

	Example: If you wanted to export all contacts that were handled between *April 13th* and *May 1st*, the **`Duration`** object in your request might look something like this:
	`"Duration": {`
	`"SearchMode": "ClientCaptureDate",`
	`"LastNDays": null,`
	`"LastNHours": null,`
	`"TimeFrame": "Custom",`
	`"StartDate": "2025-04-13T00:00:00Z",`
	`"EndDate": "2025-05-01T00:00:00Z"`
	`}`
	5. Continue following the steps in the *Bulk Export API User Guide* section and send the request.

Exporting Contact Data by Last Updated `GET /api/v2/export/lastupdated`	1. Go to Create a Job to Export Contact Data. 2. Follow the steps outlined in that section to start creating a request. 3. Find the **`Duration`** object in your request. 4. Within that object, set **`SearchMode`** to *Updated*. 5. Use the information provided in the User Guide section to set the timeframe you want the data exported from by entering a value for either **`LastNDays`**, **`LastNHours`**, or **`StartDate`** and **`EndDate`**.

Deprecated Data API Cookbook Recipe	How to Use the `POST /api/export/job` Route to Do the Same Thing

	Example: If you wanted to export all contacts that were last updated between *April 13th* and *May 1st*, the **`Duration`** object in your request might look something like this:
	`"Duration": {`
	`"SearchMode": "Updated",`
	`"LastNDays": null,`
	`"LastNHours": null,`
	`"TimeFrame": "Custom",`
	`"StartDate": "2025-04-13T00:00:00Z",`
	`"EndDate": "2025-05-01T00:00:00Z"`
	`}`
	6. Continue following the steps in the *Bulk Export API User Guide* section and send the request.

Exporting Contact Data by Contact IDs `GET /api/v2/export/searchbycontactids`	1. Go to Create a Job to Export Contact Data. 2. Follow the steps outlined in that section to start creating a request. 3. Find the **`SearchFilters`** array in your request. 4. Within this array, you will include one object for each contact that you want to export. 5. Within each object, set the **`ColumnName`** value to the column name of the Contact ID metadata field. **Note**: This should be the column name of the metadata, not the friendly name displayed in Analyze or Coach, etc. 6. Then, set **`Operator`** to = (equals), and **`Operand1`** to the contact's Contact ID value.

<table>
<tr><th>Deprecated Data API Cookbook Recipe</th><th>How to Use the <code>POST /api/export/job</code> Route to Do the Same Thing</th></tr>
<tr><td></td><td>7. Either remove the other fields that you aren't using from the object or set their value to null.

Example: If you wanted to export two specific contacts whose Contact IDs were 50625 and 28180, the <code>SearchFilters</code> array in your request might look something like this:


```
"SearchFilters": [
    {
      "ColumnName": "ID",
      "Operator": "=",
      "Operand1": "50625"
    },
    {
      "ColumnName": "ID",
      "Operator": "=",
      "Operand1": "28180"
    }
]
```


8. Continue following the steps in the Bulk Export API User Guide section and send the request.</td></tr>
<tr><td>Exporting Contact Data by Client IDs

<code>GET /api/v2/export/searchbyclientids</code></td><td>1. Go to Create a Job to Export Contact Data.
2. Follow the steps outlined in that section to start creating a request.
3. Find the <code>SearchFilters</code> array in your request.
4. Within this array, you will include one object for each contact that you want to export.
5. Within each object, set the <code>ColumnName</code> value to the column name of the Client ID metadata field.</td></tr>
</table>

Deprecated Data API Cookbook Recipe	How to Use the `POST /api/export/job` Route to Do the Same Thing

	> **Note**: This should be the column name of the metadata, not the friendly name displayed in Analyze or Coach, etc.
	
	6. Then, set **`Operator`** to = (equals), and **`Operand1`** to the contact's Client ID value.
	7. Either remove the other fields that you aren't using from the object or set their value to *null*.
	
	> **Example**: If you wanted to export two specific contacts whose Client IDs were *50625* and *28180*, the **`SearchFilters`** array in your request might look something like this:
	
	``
	"SearchFilters": [
	  {
	  "ColumnName": "ClientID",
	  "Operator": "=",
	  "Operand1": "50625"
	  },
	  {
	  "ColumnName": "ClientID",
	  "Operator": "=",
	  "Operand1": "28180"
	  }
	]
	``
	
	8. Continue following the steps in the *Bulk Export API User Guide* section and send the request.

Exporting a List of Events Using Contact IDs `POST /api/v2/export/eventsbycontactids`	1. Go to Create a Job to Export Contact Data. 2. Follow the steps outlined in that section to start creating a request. 3. Find the **`DataTypes`** array in your request.

Deprecated Data API Cookbook Recipe	How to Use the `POST /api/export/job` Route to Do the Same Thing
	4. Set the value to *Events*. 5. Find the `SearchFilters` array in your request. 6. Within this array, you will include one object for each contact that you want to export. 7. Within each object, set the `ColumnName` value to the column name of the Contact ID metadata field. > **Note**: This should be the column name of the metadata, not the friendly name displayed in Analyze or Coach, etc. 8. Then, set `Operator` to = (equals), and `Operand1` to the contact's Contact ID value. 9. Either remove the other fields that you aren't using from the object or set their value to *null*.

Deprecated Data API Cookbook Recipe	How to Use the `POST /api/export/job` Route to Do the Same Thing
	Example: If you wanted to export the events associated with two specific contacts whose Contact IDs were *50625* and *28180*, the **`DataTypes`** and **`SearchFilters`** arrays in your request might look something like this: `"DataTypes": [ "Events" ], "SearchFilters": [ { "ColumnName": "ID", "Operator": "=", "Operand1": "50625" }, { "ColumnName": "ID", "Operator": "=", "Operand1": "28180" } ]` 10. Continue following the steps in the *Bulk Export API User Guide* section and send the request.
Exporting Individual Words and Their Corresponding Information for a Single Transcript `GET /api/v2/transcript/{contactid}`	**Important**: We have created a more user-friendly option that exports each transcript and organizes it by speaker turn instead of individual word. See the row below for steps on how to use this new and improved option.
Exporting Transcripts in Bulk Using Contact IDs `POST /api/v2/export/transcriptbycontactids`	1. Go to Create a Job to Export Contact Data. 2. Follow the steps outlined in that section to start creating a request. 3. Find the **`DataTypes`** array in your request.

Deprecated Data API Cookbook Recipe	How to Use the `POST /api/export/job` Route to Do the Same Thing
	4. Set the value to *Transcripts_by_speaker*. 5. Find the **`SearchFilters`** array in your request. 6. Within this array, you will include one object for each contact whose transcript you want to export. 7. Within each object, set the **`ColumnName`** value to the column name of the Contact ID metadata field. > **Note**: This should be the column name of the metadata, not the friendly name displayed in Analyze or Coach, etc. 8. Then, set **`Operator`** to = (equals), and **`Operand1`** to the contact's Contact ID value. 9. Either remove the other fields that you aren't using from the object or set their value to *null*.

Deprecated Data API Cookbook Recipe	How to Use the `POST /api/export/job` Route to Do the Same Thing
	Example: If you wanted to export the transcripts associated with two specific contacts whose Contact IDs were *50625* and *28180*, the **`DataTypes`** and **`SearchFilters`** arrays in your request might look something like this: `"DataTypes": [ "Transcripts_by_speaker" ], "SearchFilters": [ { "ColumnName": "ID", "Operator": "=", "Operand1": "50625" }, { "ColumnName": "ID", "Operator": "=", "Operand1": "28180" } ]` 10. Continue following the steps in the *Bulk Export API User Guide* section and send the request.
Exporting a List of Contact AI Summaries Using Contact IDs **Note**: The Bulk Export system will only export AI-generated summaries. It will not export the topic-based Hit/Missed Summaries or Text Summaries (generated by contact summary templates) for each contact.	1. Go to Create a Job to Export Contact Data. 2. Follow the steps outlined in that section to start creating a request. 3. Find the **`DataTypes`** array in your request. 4. Set the value to *Ai_summaries*. 5. Find the **`SearchFilters`** array in your request. 6. Within this array, you will include one object for each contact whose summary you want to export.

Deprecated Data API Cookbook Recipe	How to Use the `POST /api/export/job` Route to Do the Same Thing

`POST /api/v2/export/summarizationsbycontactids`	7. Within each object, set the **`ColumnName`** value to the column name of the Contact ID metadata field.
	
	> **Note**: This should be the column name of the metadata, not the friendly name displayed in Analyze or Coach, etc.
	
	8. Then, set **`Operator`** to = (equals), and **`Operand1`** to the contact's Contact ID value.
	
	9. Either remove the other fields that you aren't using from the object or set their value to *null*.
	
	> **Example**: If you wanted to export the AI summaries associated with two specific contacts whose Contact IDs were *50625* and *28180*, the **`DataTypes`** and **`SearchFilters`** arrays in your request might look something like this:
	
	`"DataTypes": [`
	`  "Ai_summaries"`
	`  ],`
	`"SearchFilters": [`
	`  {`
	`  "ColumnName": "ID",`
	`  "Operator": "=",`
	`  "Operand1": "50625"`
	`  },`
	`  {`
	`  "ColumnName": "ID",`
	`  "Operator": "=",`
	`  "Operand1": "28180"`
	`  }`
	`]`
	
	10. Continue following the steps in the *Bulk Export API User Guide* section and send the request.

Deprecated Data API Cookbook Recipe	How to Use the `POST /api/export/job` Route to Do the Same Thing

Getting a List of Category Hits for a Specific Category
 	
`GET /api/v2/export/lastupdated`
	1. Go to Create a Job to Export Contact Data.
	2. Follow the steps outlined in that section to start creating a request.
	3. Find the **`SearchFilters`** array in your request.
	4. Within the **`SearchFilters`** array, include one object for the category whose hits you want to export.
	5. Within that object, set **`CategoryId`** to the ID of the category.
	6. Then, set **`Operator`** to = (equals).
	7. Use the information provided in the User Guide section to set the timeframe you want the data exported from by entering a value for either **`LastNDays`**, **`LastNHours`**, or **`StartDate`** and **`EndDate`**.
	8. Either remove the other fields that you aren't using from the object or set their value to *null*.
	
	Example: If you wanted to export all contacts that had category hits for your *Empathy* category (whose Category ID is *12345*), the **`SearchFilters`** array in your request might look something like this:
	
	`"SearchFilters": [`
	`  {`
	`  "CategoryId": 12345,`
	`  "Operator": "="`
	`  }`
	`]`
	
	9. Continue following the steps in the *Bulk Export API User Guide* section and send the request.

Getting a List of Score Results for a	1. Go to Create a Job to Export Contact Data.

Deprecated Data API Cookbook Recipe	How to Use the `POST /api/export/job` Route to Do the Same Thing
Specific Score `GET /api/v2/export/lastupdated`	2. Follow the steps outlined in that section to start creating a request. 3. Find the **`SearchFilters`** array in your request. 4. Within the **`SearchFilters`** array, include one object for each range of the score whose results you want to export. 5. Within each object, set **`ScoreId`** to the ID of the score. 6. Then, set **`Operator`** to = (equals) and **`Operand1`** to the score range name (or score range value) you want to export results for. 7. Use the information provided in the User Guide section to set the timeframe you want the data exported from by entering a value for either **`LastNDays`**, **`LastNHours`**, or **`StartDate`** and **`EndDate`**. 8. Either remove the other fields that you aren't using from the object or set their value to *null*.

<table>
<tr><th>Deprecated Data API Cookbook Recipe</th><th>How to Use the <code>POST /api/export/job</code> Route to Do the Same Thing</th></tr>
<tr><td></td><td>

> **Example**: If you wanted to export all contacts that fell into the *Acceptable* or *Good* score range for your *Customer Satisfaction* score (whose Score ID is *12345*), the **`SearchFilters`** array in your request might look something like this:

```
"SearchFilters": [
    {
      "ScoreId": 12345,
      "Operator": "=",
      "Operand1": "Acceptable"
    },
    {
      "ScoreId": 12345,
      "Operator": "=",
      "Operand1": "Good"
    }
]
```

9. Continue following the steps in the *Bulk Export API User Guide* section and send the request.

</td></tr>
<tr><td>

Downloading the Audio for a Specific Contact

> **Note**: If you attempt to export a video contact, the Bulk Export system will export the audio associated with the video contact only. The video file will not be exported.

`GET /api/v2/downloadaudio/{contactId}/{format}`

</td><td>

1. Go to Create a Job to Export Contact Data.
2. Follow the steps outlined in that section to start creating a request.
3. Find the **`DataTypes`** array in your request.
4. Set the value to *Audio*.
5. Find the **`SearchFilters`** array in your request.
6. Within this array, you will include one object for the contact whose audio you want to export.
7. Within the object, set the **`ColumnName`** value to the column name of the Contact ID metadata field.

</td></tr>
</table>

Deprecated Data API Cookbook Recipe	How to Use the `POST /api/export/job` Route to Do the Same Thing

	> **Note**: This should be the column name of the metadata, not the friendly name displayed in Analyze or Coach, etc.
	
	8. Then, set **`Operator`** to = (equals), and **`Operand1`** to the contact's Contact ID value.
	9. Either remove the other fields that you aren't using from the object or set their value to *null*.
	
	> **Example**: If you wanted to export the audio associated with the contact whose Contact ID is *50625*, the **`DataTypes`** and **`SearchFilters`** arrays in your request might look something like this:
	
	`"DataTypes": [`
	`  "Audio"`
	`  ],`
	`"SearchFilters": [`
	`  {`
	`  "ColumnName": "ID",`
	`  "Operator": "=",`
	`  "Operand1": "50625"`
	`  }`
	`]`
	
	10. Continue following the steps in the *Bulk Export API User Guide* section and send the request.

Downloading the Transcript for a Specific Contact `GET /api/v2/downloadtranscript/{contactid}`	1. Go to Create a Job to Export Contact Data. 2. Follow the steps outlined in that section to start creating a request. 3. Find the **`DataTypes`** array in your request. 4. Set the value to *Transcripts_by_speaker*.

Deprecated Data API Cookbook Recipe	How to Use the `POST /api/export/job` Route to Do the Same Thing

	5. Find the **`SearchFilters`** array in your request.
	6. Within this array, you will include one object for the contact whose transcript you want to export.
	7. Within the object, set the **`ColumnName`** value to the column name of the Contact ID metadata field.
	> **Note**: This should be the column name of the metadata, not the friendly name displayed in Analyze or Coach, etc.
	8. Then, set **`Operator`** to = (equals), and **`Operand1`** to the contact's Contact ID value.
	9. Either remove the other fields that you aren't using from the object or set their value to *null*.
	> **Example**: If you wanted to export the transcript associated with the contact whose Contact ID is *28180*, the **`DataTypes`** and **`SearchFilters`** arrays in your request might look something like this:
	`"DataTypes": [`
	`"Transcripts_by_speaker"`
	`],`
	`"SearchFilters": [`
	`{`
	`"ColumnName": "ID",`
	`"Operator": "=",`
	`"Operand1": "28180"`
	`}`
	`]`
	10. Continue following the steps in the *Bulk Export API User Guide* section and send the request.

> **Important**: Don't forget that the Bulk Export system has even more export types and filter options you can use to target specific data in your system.
>
> Check out Export Data Types and SearchFilters Basics to learn even more ways you can customize your exports.